

AGNTCon + MCPCon Europe 2026
@RAI Amsterdam Convention Centre, Amsterdam, Netherlands 

Potential Issues for Cross-domain Multi-hop API Calls and Their Solution Proposal

Takashi Norimatsu
Open Source Program Office (OSPO)
Hitachi, Ltd.

Date
September 18, 2026

Self Introduction

Takashi Norimatsu Ph.D. ([tnorimat](#) in GitHub)

- Keycloak* maintainer (since Oct 2021),
- Technical lead of Keycloak community "[OAuth SIG](#)",
- Certified Information Systems Security Professional (CISSP)
- Chief OSS Specialist, Hitachi, Ltd., Japan 

*: Identity and Access Management (IAM) open-source software, CNCF incubating project (<https://github.com/keycloak/keycloak>)

Self Introduction

Contributing security features to Keycloak since 2017:

- W3C Web Authentication API support (Passkeys)
- IETF standards:
RFC 7636 PKCE, RFC 8705 OAuth MTLS, RFC 9126 PAR, RFC 8032/8037 EdDSA,
RFC 9207 OAuth2 Authz Server Issuer Identification, RFC 9449 DPoP,
RFC 8414 OAuth 2.0 Authorization Server Metadata
Internet Draft: The OAuth 2.1 Authorization Framework
Internet Draft: OAuth Client ID Metadata Document (CIMD)
- OpenID Foundation standards:
FAPI 1.0 Baseline, FAPI 1.0 Advanced,
FAPI 2.0 Security Profile, FAPI 2.0 Message Signing,
Client Initiated Backchannel Authentication, FAPI-CIBA
- Agentic AI Foundation (AAIF) Model Context Protocol (MCP) standards:
MCP 2025-06-18, 2025-11-25, 2026-07-28

Self Introduction

Contributing conformance tests for Authorization Server since 2026:

- MCP: conformance

MCP 2025-11-28, EMA/ID-JAG (in progress)

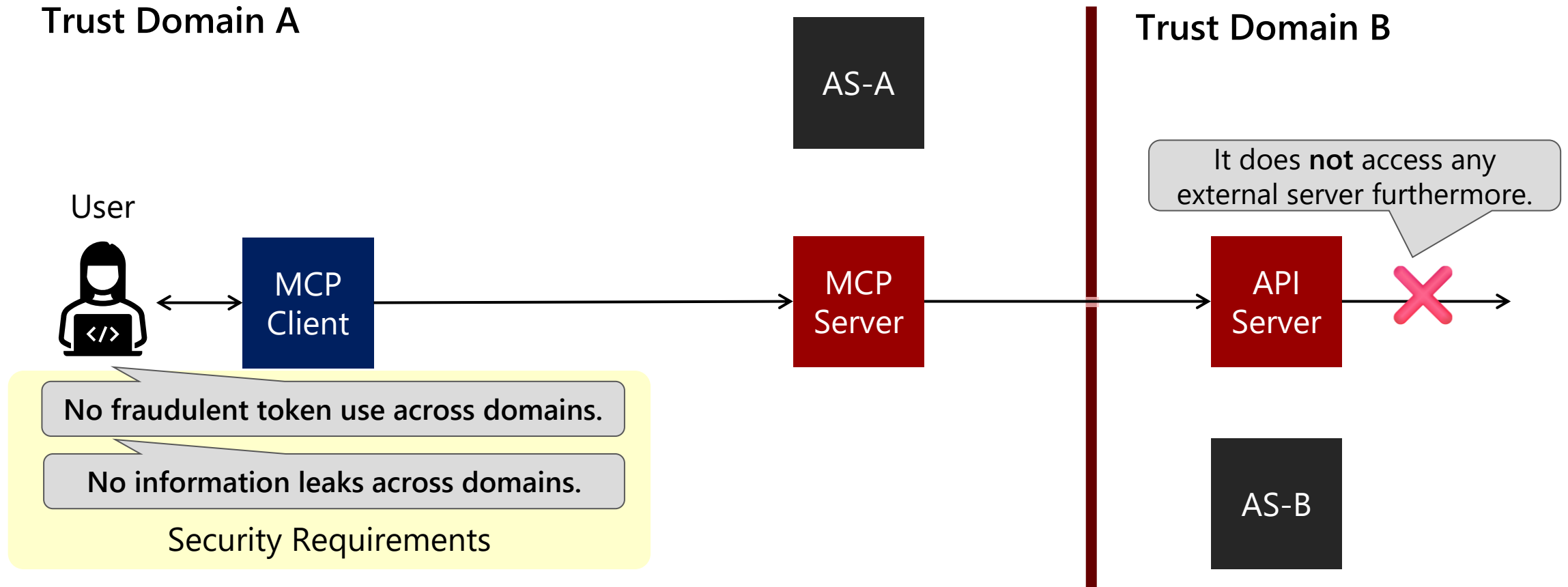
Contents

1. Methods of Cross-domain API Calls
2. Fraudulent Access Token Use
3. Information Leaks
4. User Swapping

Contents

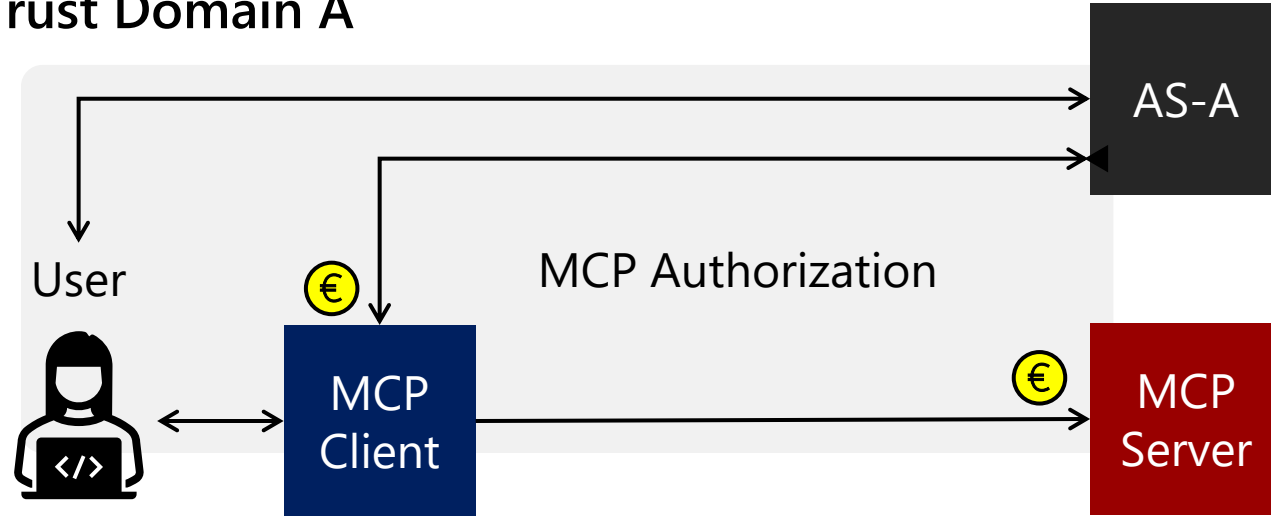
1. Methods of Cross-domain API Calls
2. Fraudulent Access Token Use
3. Information Leaks
4. User Swapping

Cross-Domain API Calls: Scope

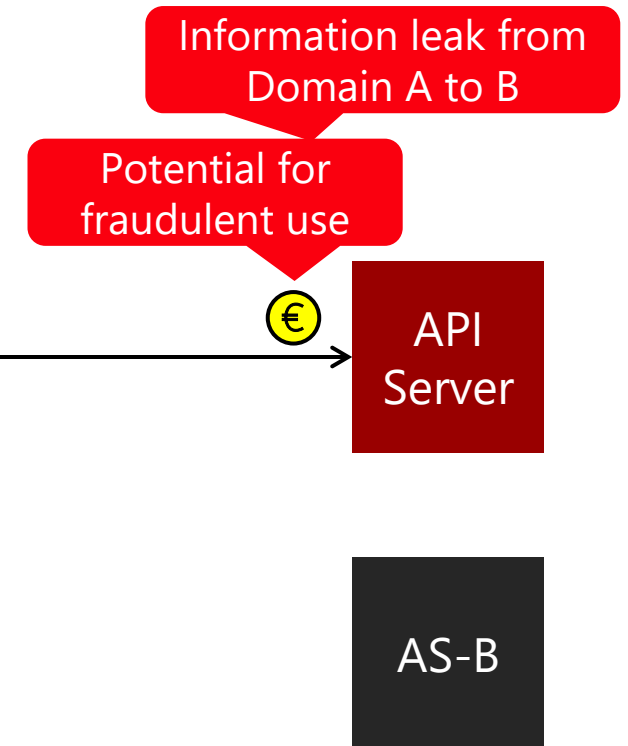


Forwarding

Trust Domain A



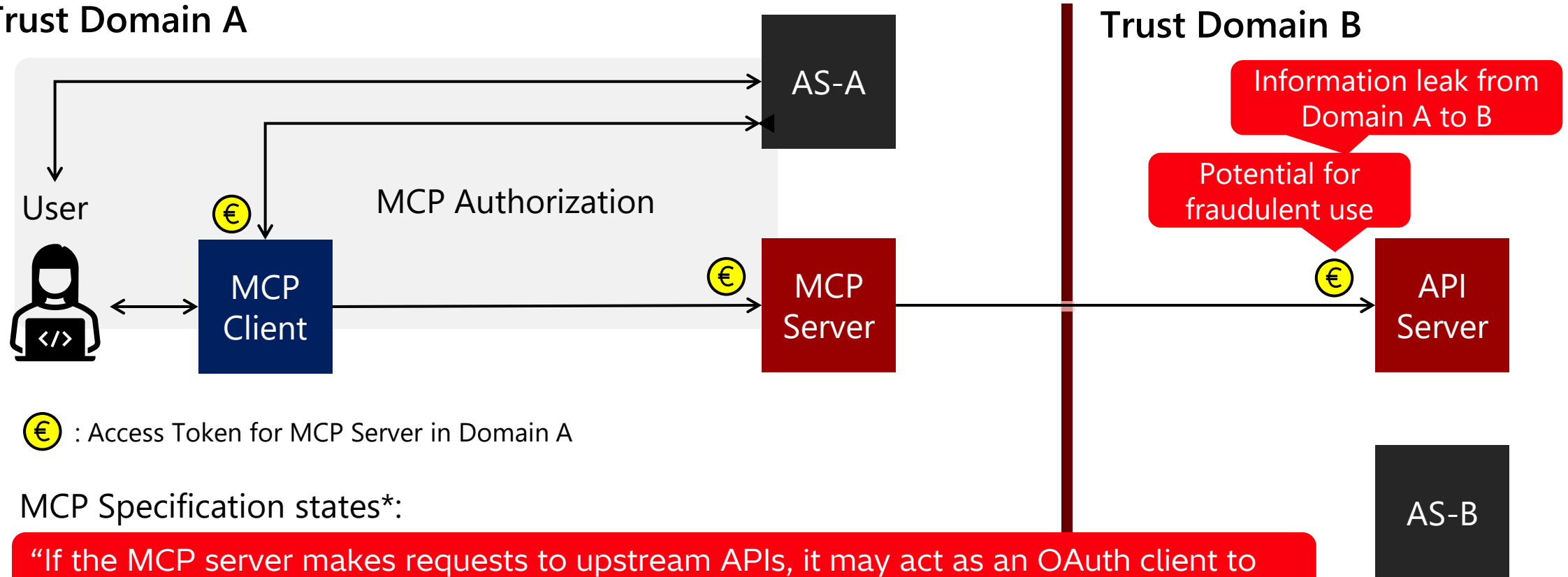
Trust Domain B



€ : Access Token for MCP Server in Domain A

Forwarding

Trust Domain A



€ : Access Token for MCP Server in Domain A

MCP Specification states*:

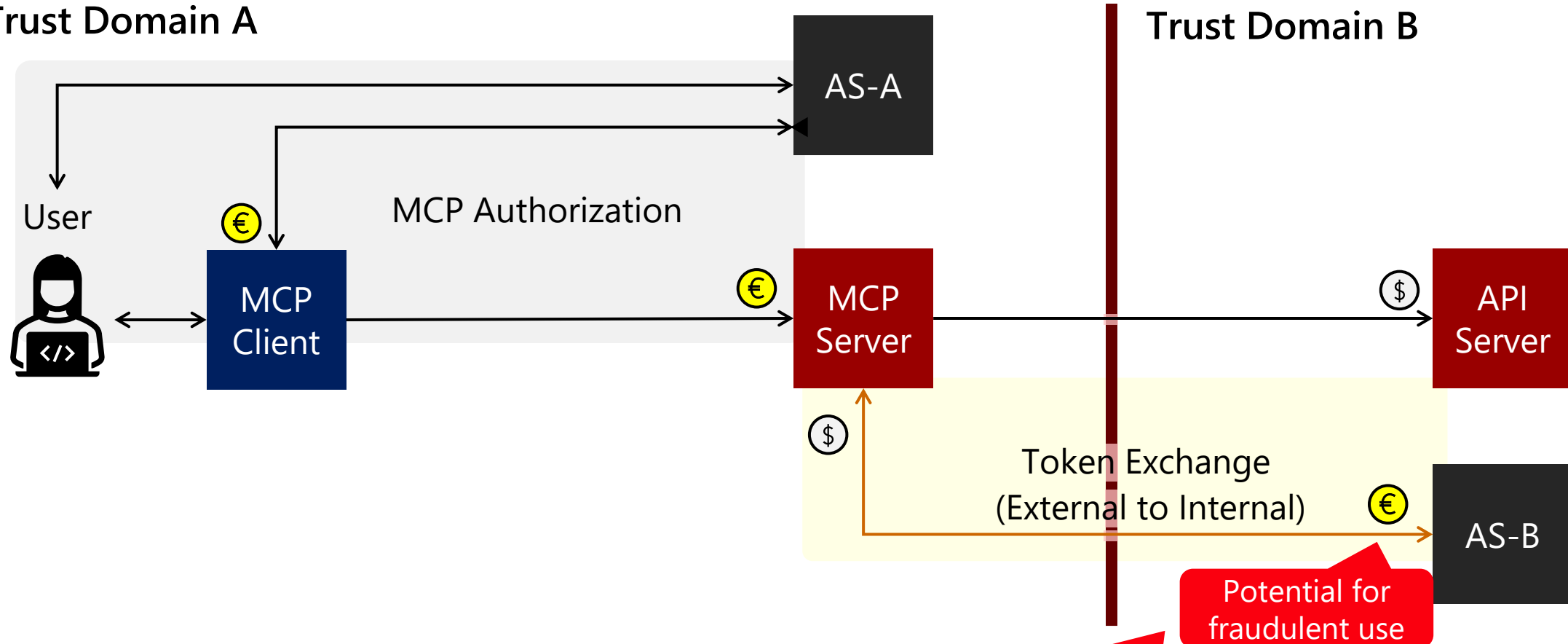
“If the MCP server makes requests to upstream APIs, it may act as an OAuth client to them. The access token used at the upstream API is a separate token, issued by the upstream authorization server. The MCP server MUST NOT pass through the token it received from the MCP client.”

*: <https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization/security-considerations#access-token-privilege-restriction>

Token Exchange (External to Internal)

Trust Domain A

Trust Domain B



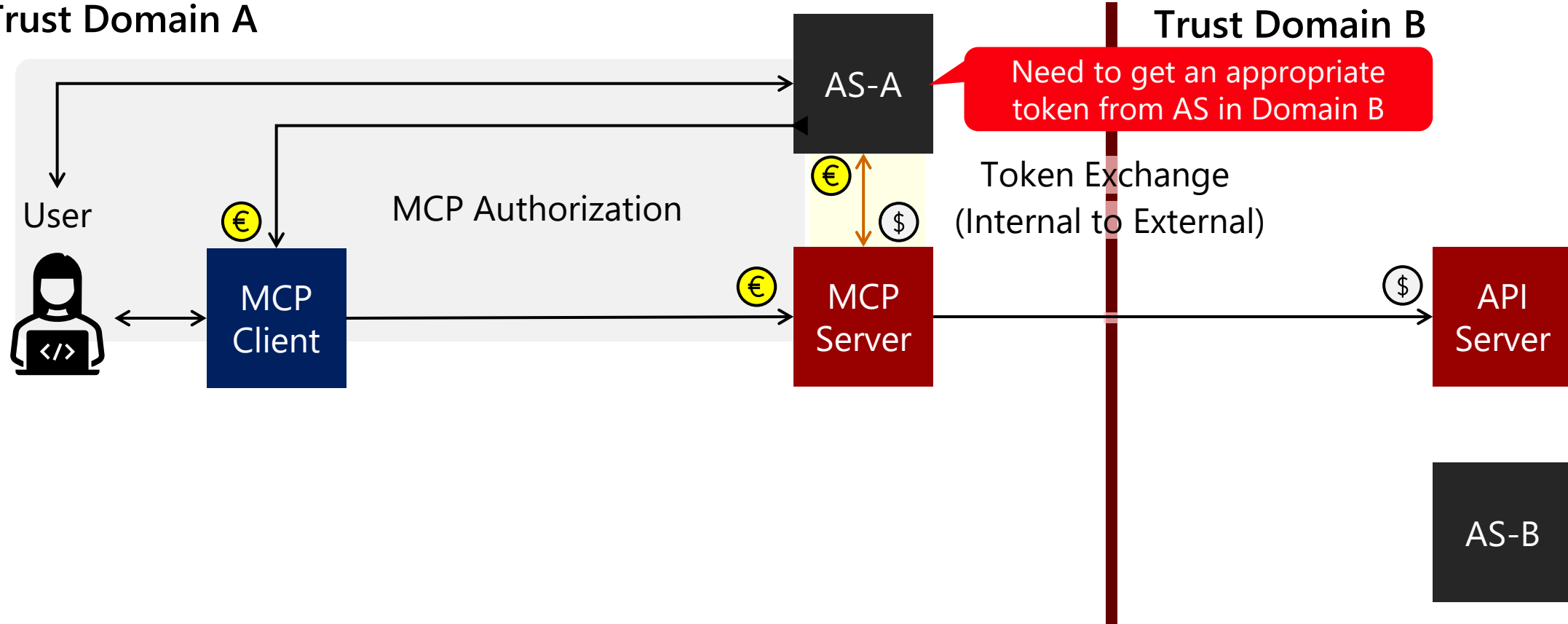
€ : Access Token for MCP Server in Domain A

\$: Access Token for API Server in Domain B

Token Exchange (Internal to External)

Trust Domain A

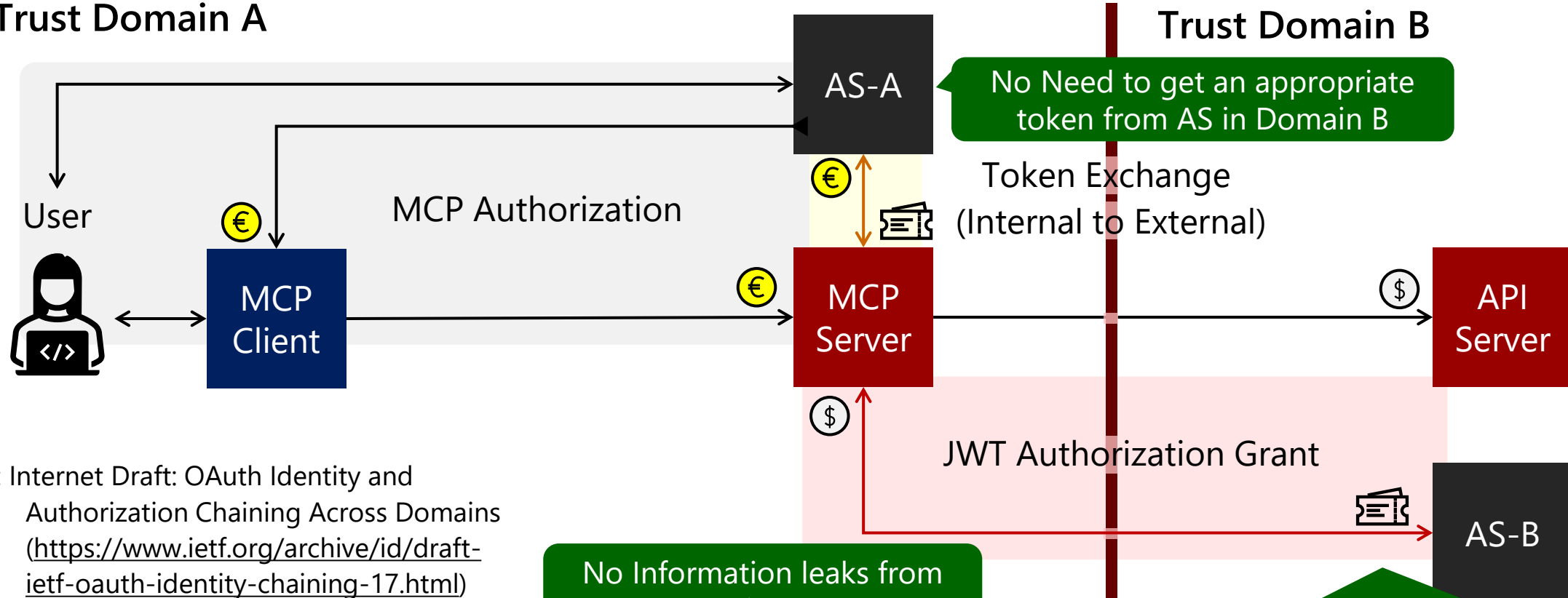
Trust Domain B



OAuth Identity and Authorization Chaining Across Domains*

Trust Domain A

Trust Domain B



*: Internet Draft: OAuth Identity and Authorization Chaining Across Domains (<https://www.ietf.org/archive/id/draft-ietf-oauth-identity-chaining-17.html>)



: Access Token for MCP Server in Domain A

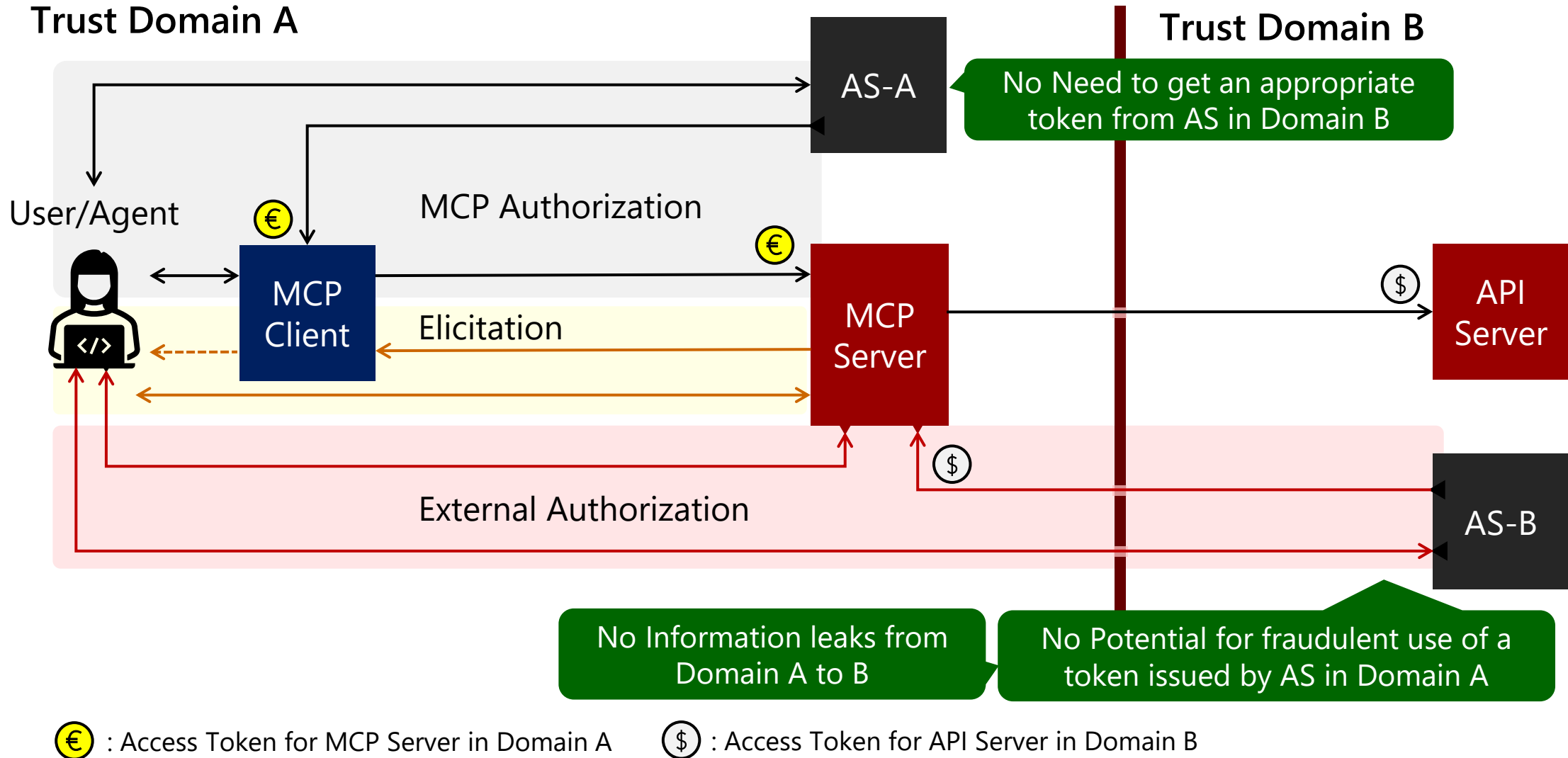


: JWT-formatted Assertion from Domain A to B without sensitive info



: Access Token for API Server in Domain B

MCP Elicitation in URL Mode



Comparison

Characteristics		MCP Elicitation in URL Mode	OAuth Identity and Authorization Chaining Across Domains
User Interaction	A user interacts on AS in Domain B	Yes	No
User Consent	A user gives consent on AS in Domain B	Explicitly, Directly	Implicitly, Indirectly (out-of-band)
Identity Chaining*	AS in Domain A knows which user ID corresponds to which one in Domain B or vice versa	No	Yes
Authorization Chaining*	AS in Domain B can take over authorization information from AS in Domain A	Limited	Yes

*: Claims Transcription (<https://www.ietf.org/archive/id/draft-ietf-oauth-identity-chaining-17.html#section-2.5>)

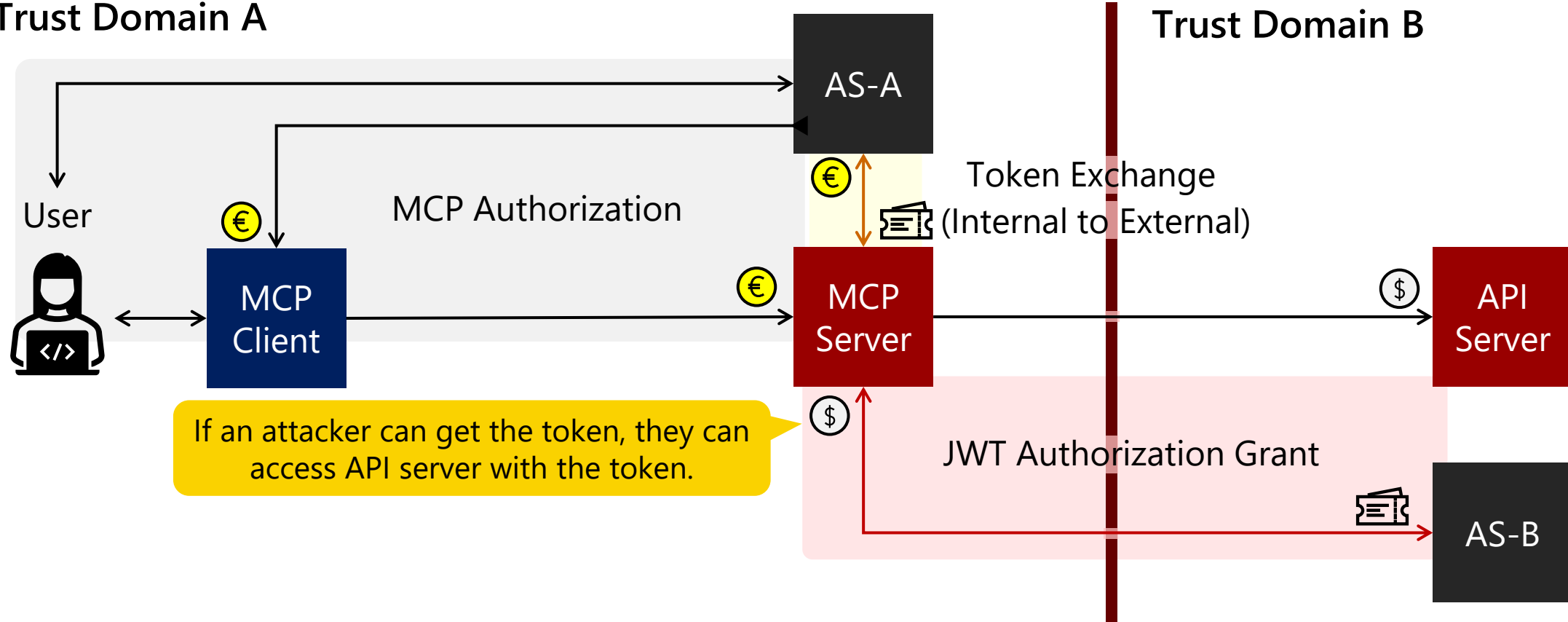
Contents

1. Methods of Cross-domain API Calls
2. **Fraudulent Access Token Use**
3. Information Leaks
4. User Swapping

OAuth Identity and Authorization Chaining Across Domains

Trust Domain A

Trust Domain B



: Access Token for MCP Server in Domain A

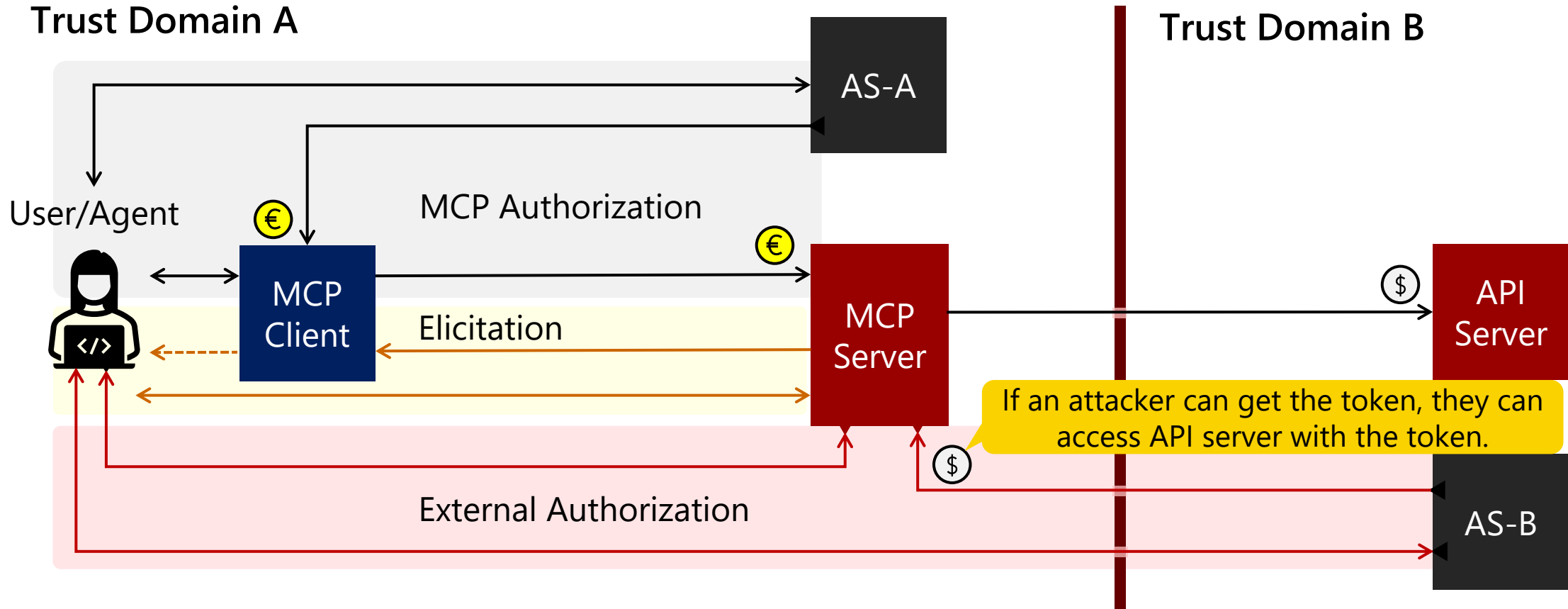


: JWT-formatted Assertion from Domain A to B without sensitive info



: **Bearer** Access Token for API Server in Domain B

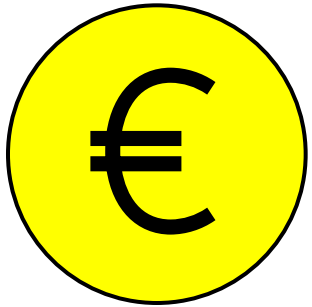
MCP Elicitation in URL Mode



€ : Access Token for MCP Server in Domain A

\$: **Bearer** Access Token for API Server in Domain B

Risk Mitigation and/or Aversion



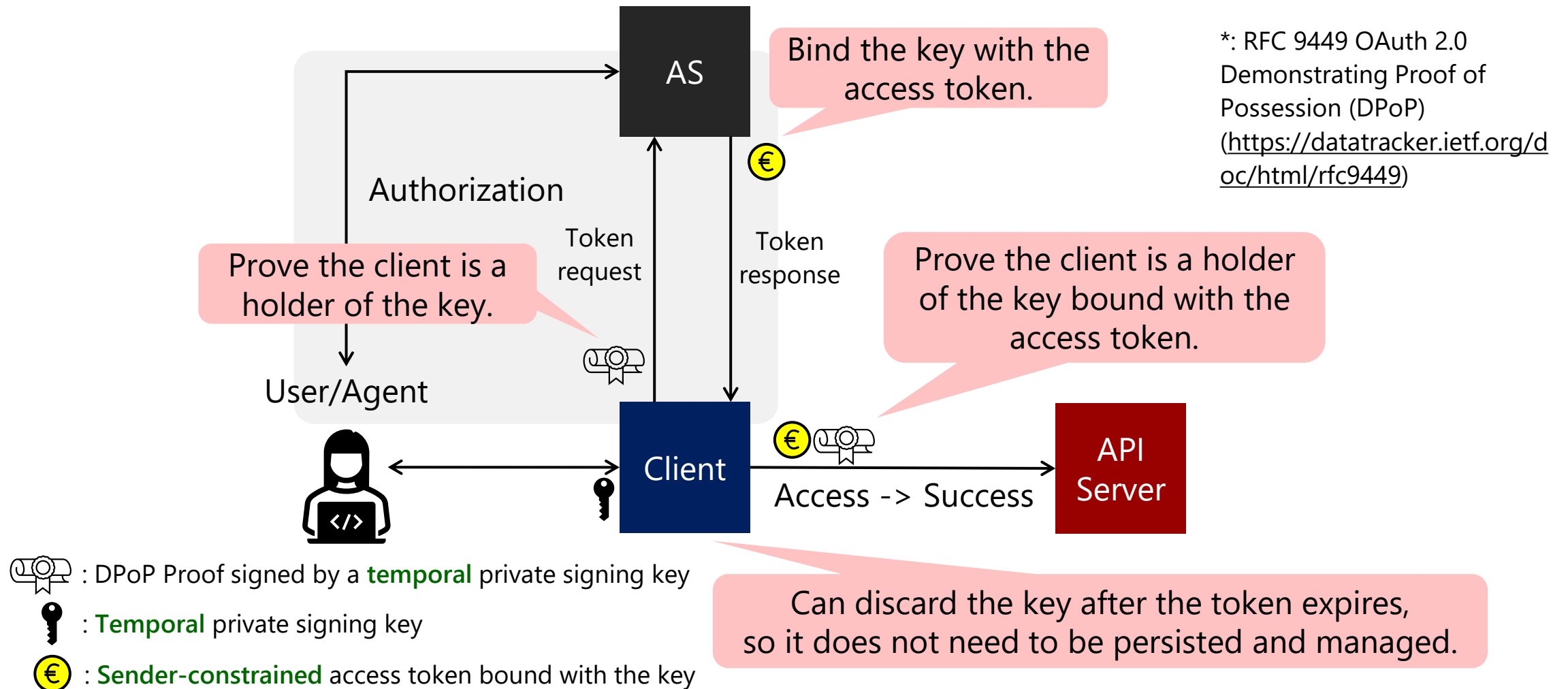
Risk Mitigation:

- Shortening token effective time
- Narrowing token scope

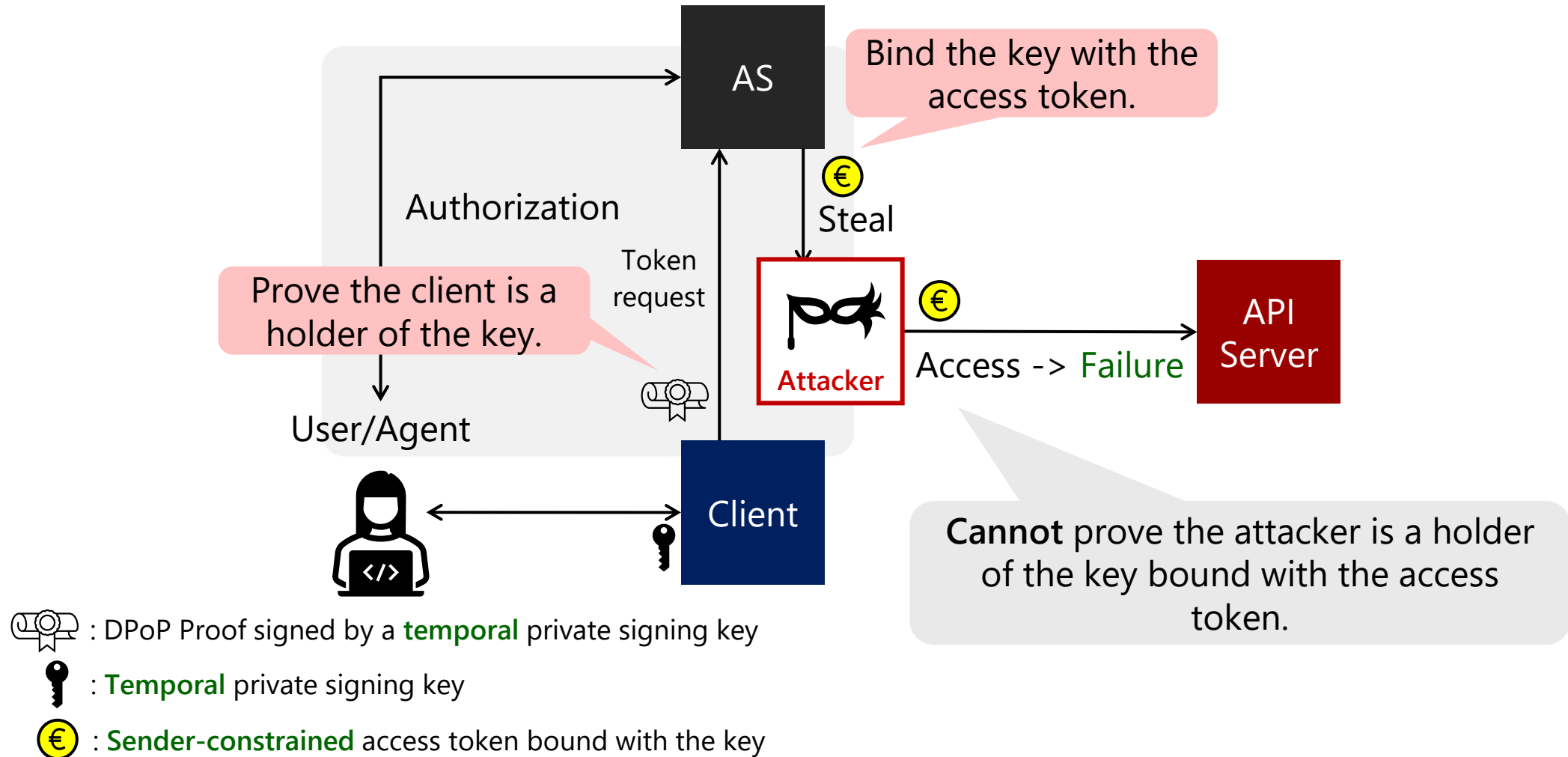
Risk Aversion:

- Sender-constrained token

Sender-Constrained Access Token - DPOP*



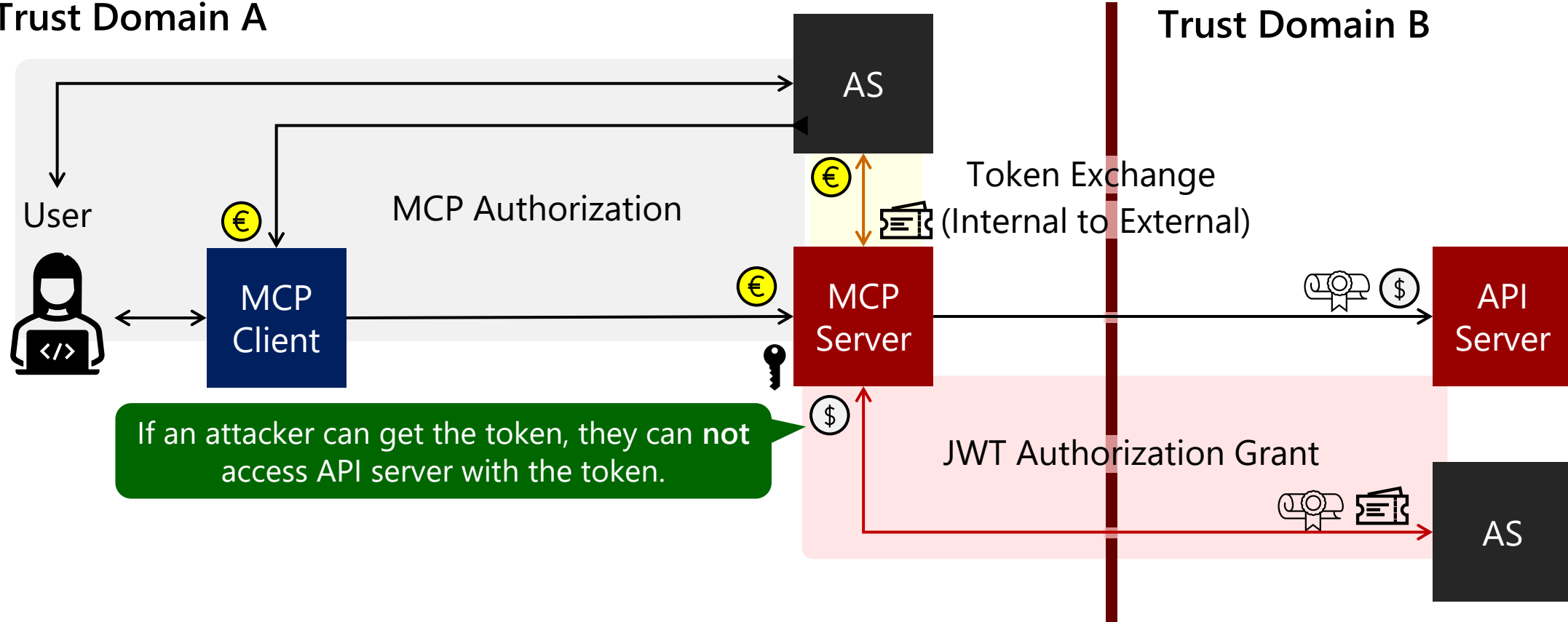
Sender-Constrained Access Token - DPoP



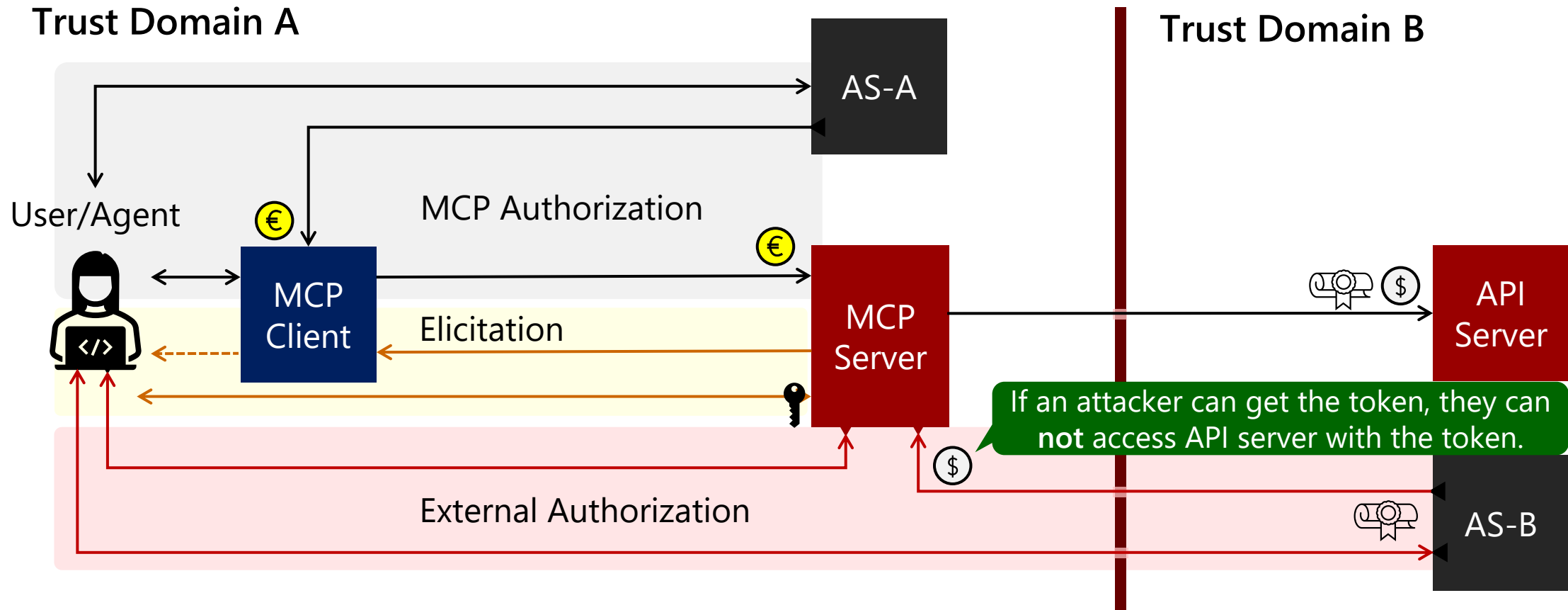
OAuth Identity and Authorization Chaining Across Domains + DPoP

Trust Domain A

Trust Domain B



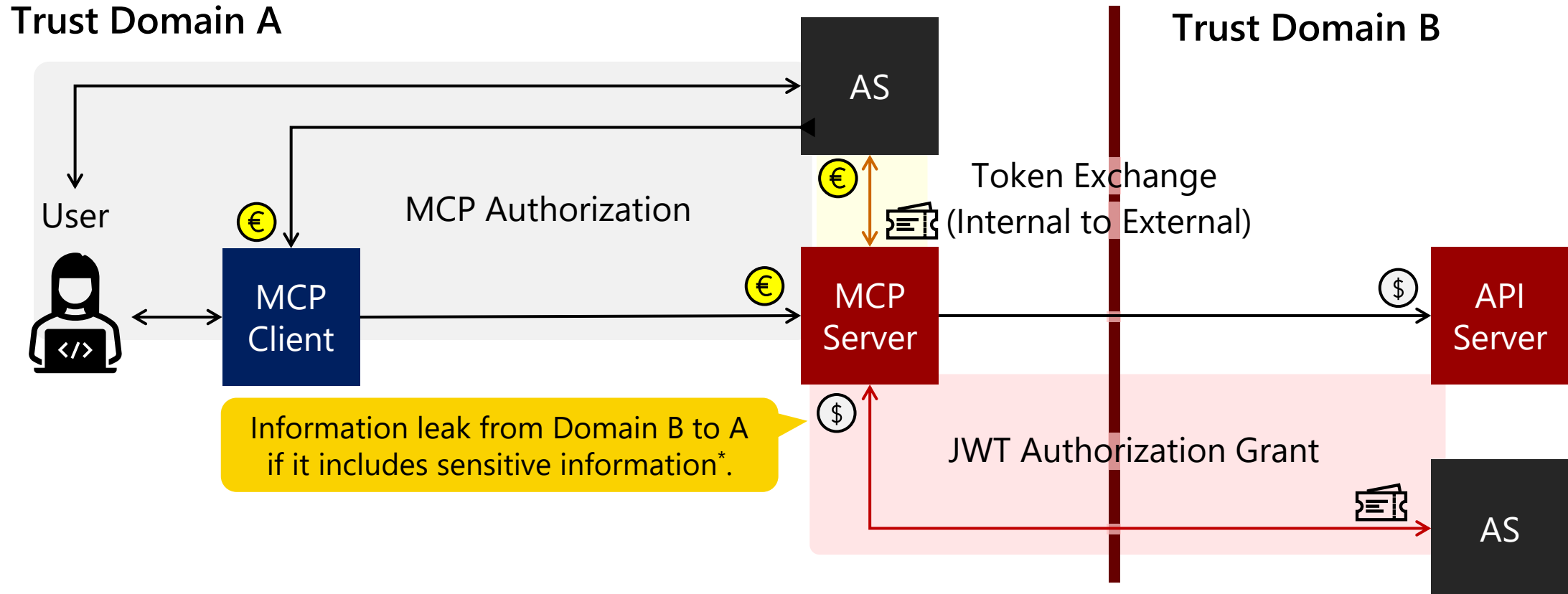
MCP Elicitation in URL Mode + DPoP



Contents

1. Methods of Cross-domain API Calls
2. Fraudulent Access Token Use
- 3. Information Leaks**
4. User Swapping

OAuth Identity and Authorization Chaining Across Domains



*: Ex. if the API server an initiate financial transaction, its transaction number, information on the bank account, etc.



: Access Token for MCP Server in Domain A

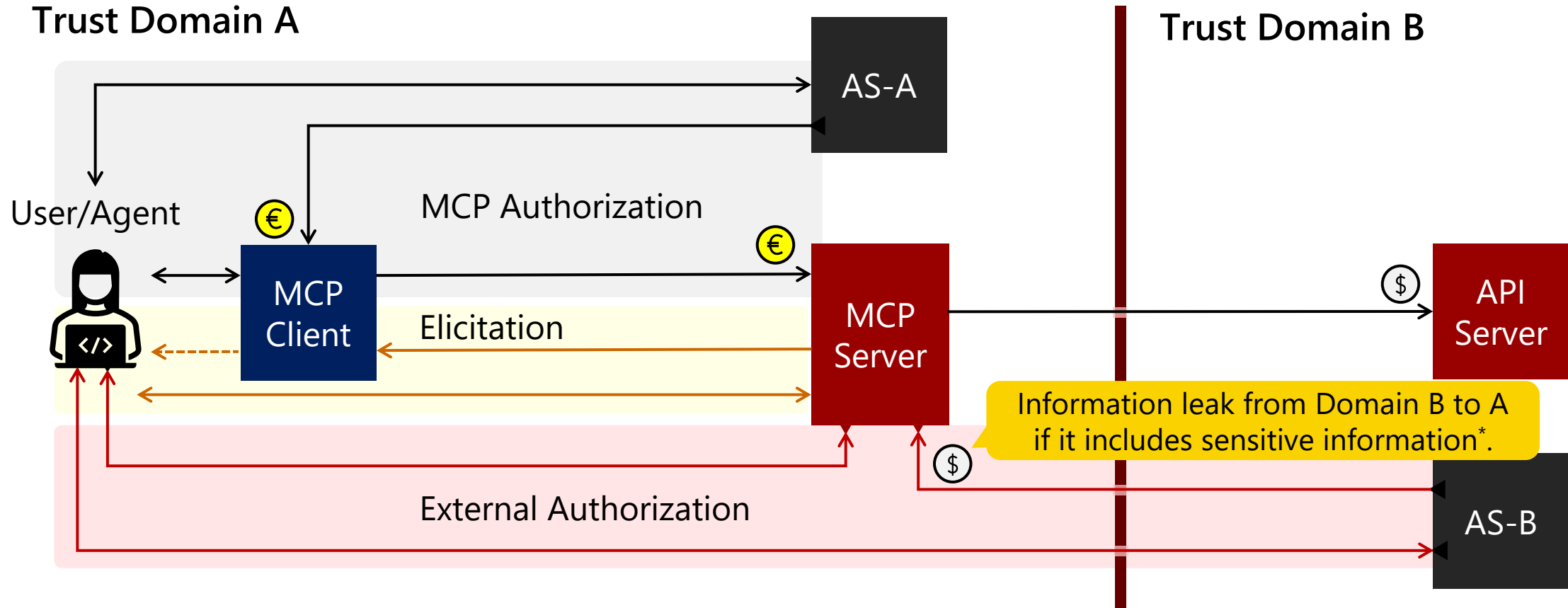


: JWT-formatted Assertion from Domain A to B without sensitive info



: Access Token for API Server in Domain B

MCP Elicitation in URL Mode

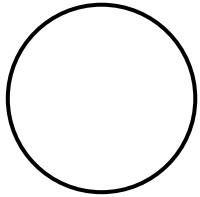


*: Ex. if the API server can initiate a financial transaction, its transaction number, information on the bank account, etc.

€ : Access Token for MCP Server in Domain A

\$: Access Token for API Server in Domain B

Token Types That Can Prevent Information Leaks



Handle/Reference type token:

- The token does not include any information.
- Its intended receiver can only extract them by token introspection^{*1}.



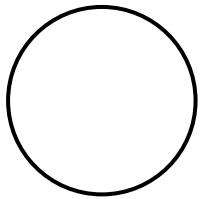
Signed and Encrypted JWT-formatted Assertion/Self-contained type token:

- The token includes all Information.
- The token is signed and encrypted by its issuer^{*2}.
- The token can only be decrypted by its intended receiver^{*2}.

*1: OAuth 2.0 Token Introspection (<https://datatracker.ietf.org/doc/html/rfc7662>)

*2: By applying Key Encryption defined by RFC 7516 and 7518, a token itself is encrypted by a shared key named Content Encryption Key (CEK) in symmetric algorithm and the CEK is encrypted by a public key named Key Encryption Key (KEK) and decrypted by its private key in asymmetric algorithm.

Token Types That Can Prevent Information Leaks



Handle/Reference type token:

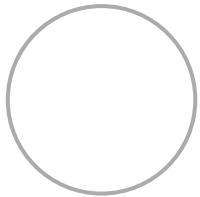
- **Pros:** No information included.
 - **Cons:** Need token introspection.
 - **Cons:** Cannot know who issue the token.
- ➔ No information leaked.
 - ➔ Response time increased.
 - ➔ Cannot support multiple AS.



Signed and Encrypted JWT-formatted Assertion/Self-contained type token:

- **Pros:** No need token introspection.
 - **Pros:** Can know who issue the token.
 - **Cons:** If decryption key is stolen
 - **Cons:** Need decryption.
- ➔ Response time not increased.
 - ➔ Can support multiple AS.
 - ➔ Information leaked.
 - ➔ Response time increased.

Token Types That Can Prevent Information Leaks - Alternative



Handle/Reference type token:

- The token does not include any information.
- Its intended receiver can only extract them by token introspection.



Lightweight token*:

- The token includes only Information on token itself.
- Other information can be extracted by token introspection*.
- The token is signed by its issuer.



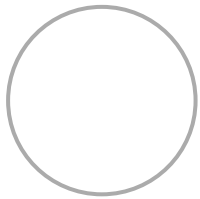
Signed and Encrypted JWT-formatted Assertion/Self-contained type token:

- The token includes all Information.
- The token is signed and encrypted by its issuer.
- The token can only be decrypted by its intended receiver.

*: As far as I know, there is no formal definition. I quoted it from Keycloak.

(https://www.keycloak.org/docs/latest/server_admin/index.html#_using_lightweight_access_token)

Token Types That Can Prevent Information Leaks



Handle/Reference type token:

- **Pros:** No information included.
 - **Cons:** Need token introspection.
 - **Cons:** Cannot know who issue the token.
- ➔ No information leaked.
 - ➔ Response time increased.
 - ➔ Cannot support multiple AS.



Lightweight token:

- **Pros:** Can know who issue the token.
 - **Pros:** Can verify token validity
 - **Cons:** Need token introspection.
- ➔ Can support multiple AS.
 - ➔ Avoid unnecessary introspection.
 - ➔ Response time increased.



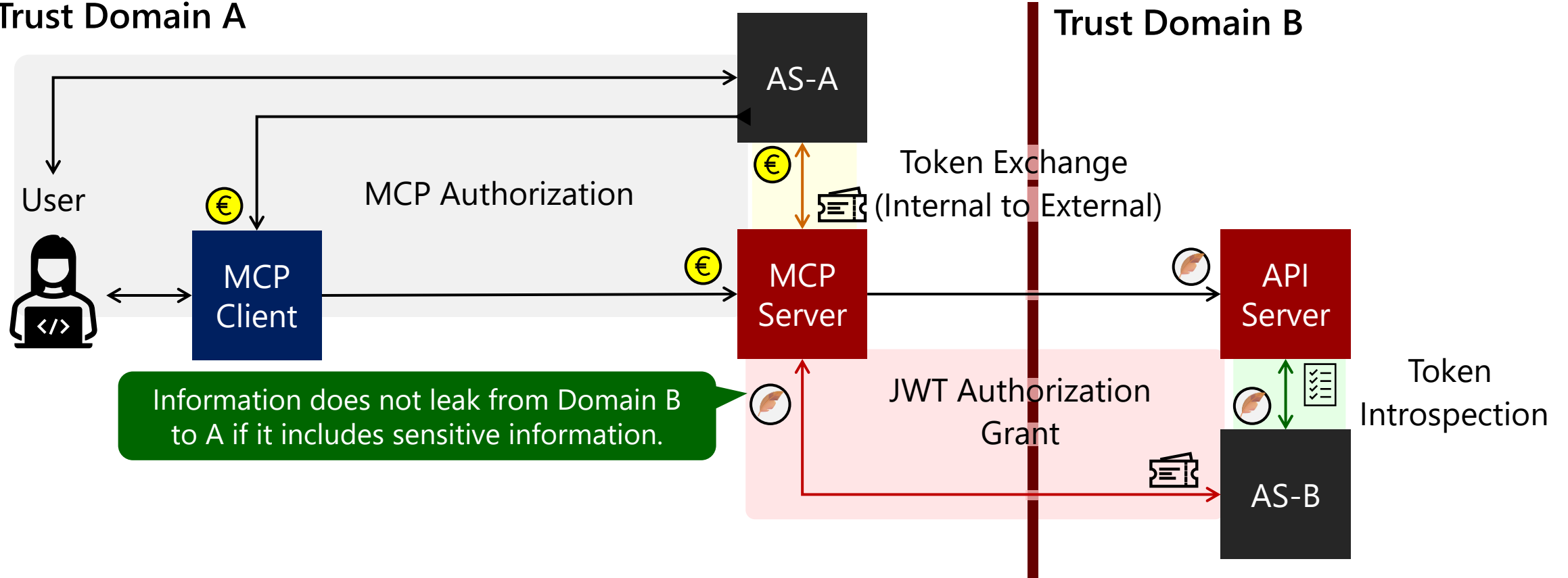
Signed and Encrypted JWT-formatted Assertion/Self-contained type token:

- **Pros:** No need token introspection.
 - **Pros:** Can know who issue the token.
 - **Cons:** If decryption key is stolen
 - **Cons:** Need decryption.
- ➔ Response time not increased.
 - ➔ Can support multiple AS.
 - ➔ Information leaked.
 - ➔ Response time increased.

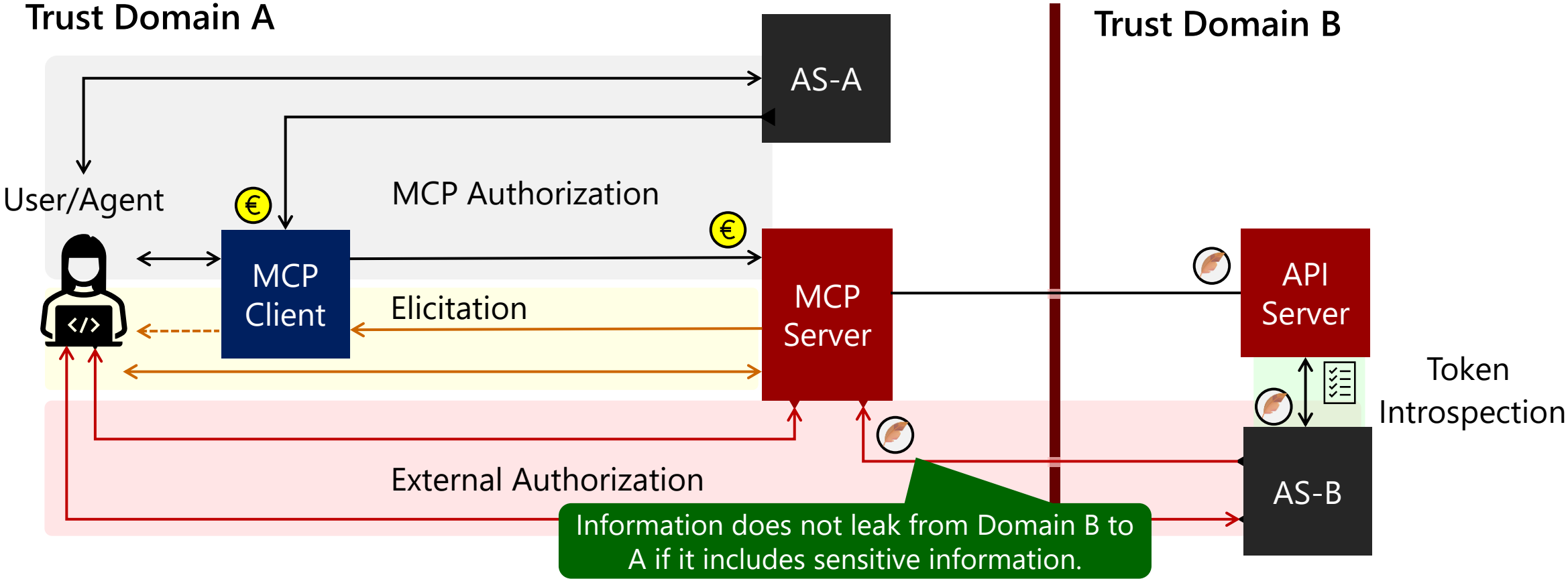
OAuth Identity and Authorization Chaining Across Domains

Trust Domain A

Trust Domain B



MCP Elicitation in URL Mode

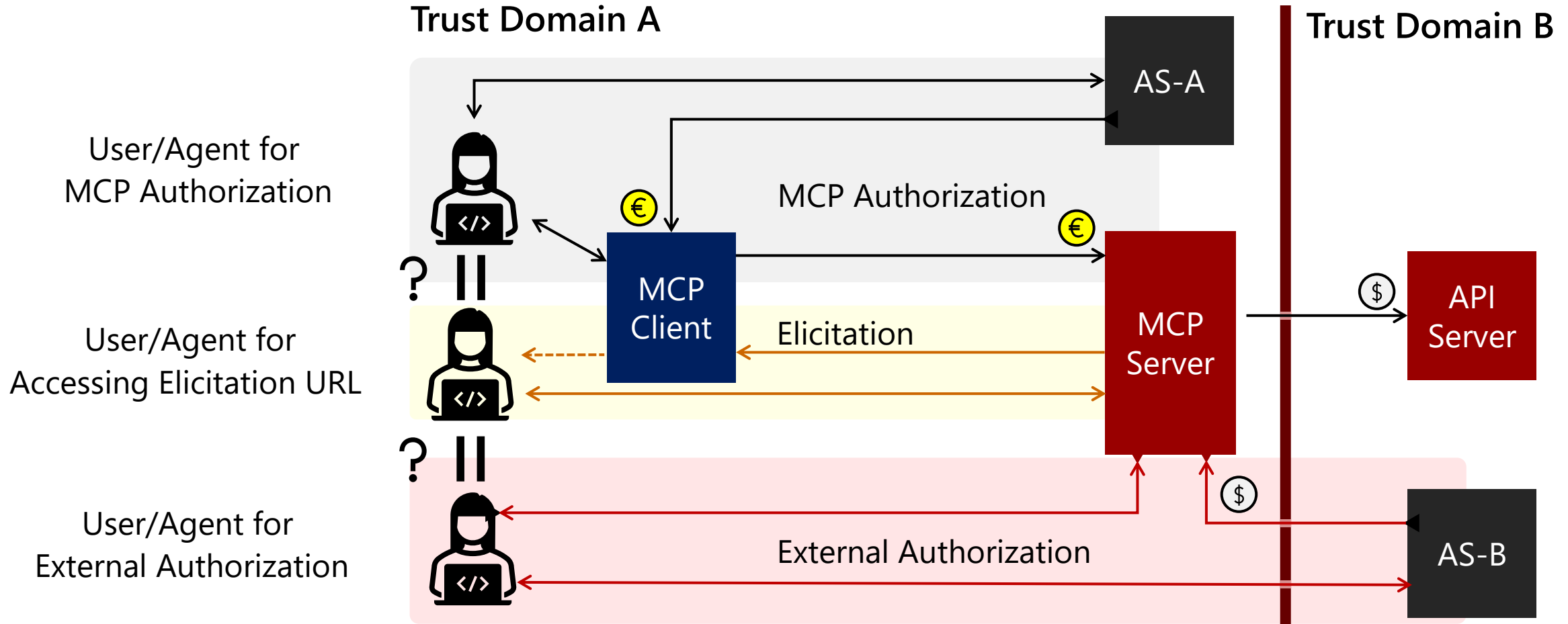


- € : Access Token for MCP Server in Domain A
- ☞ : Identity and authorization sensitive info in Domain B
- ☞ : Lightweight access token for API Server in Domain B

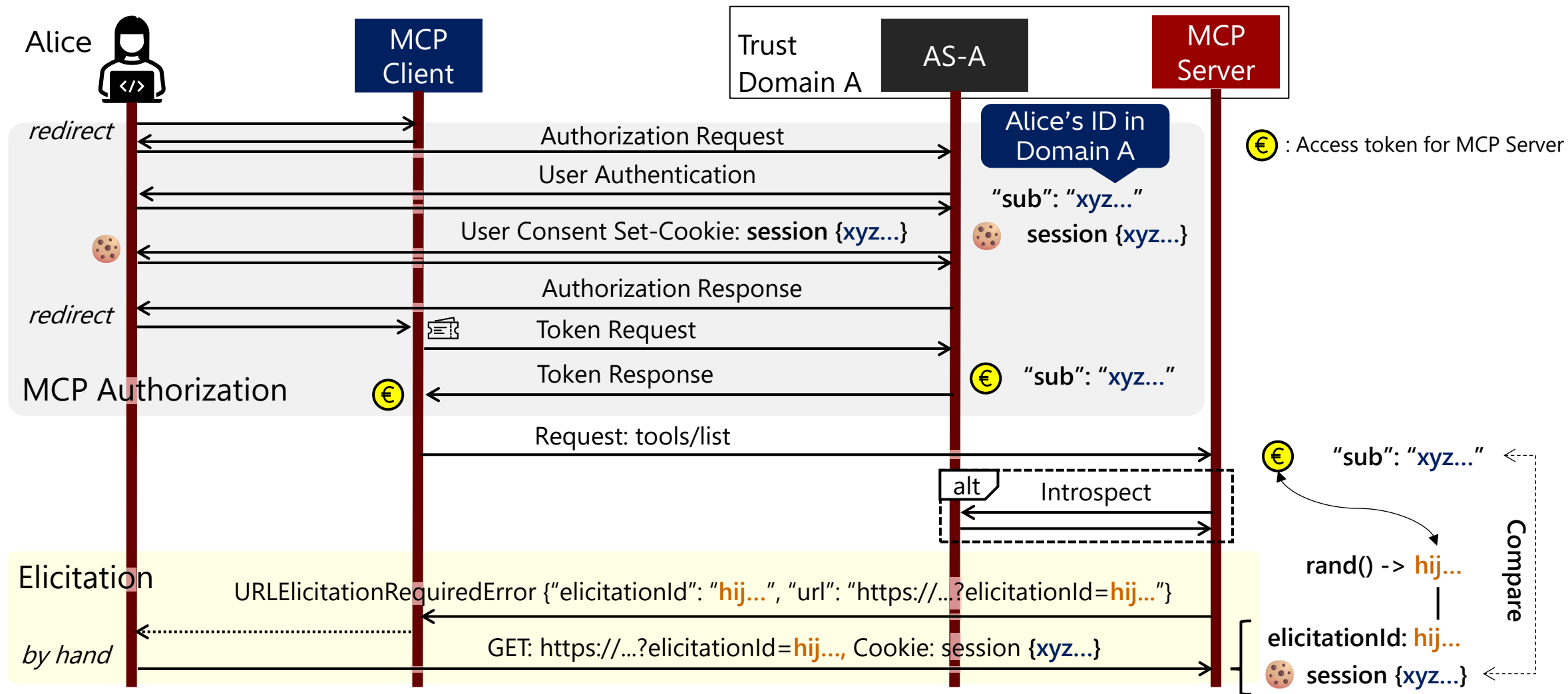
Contents

1. Methods of Cross-domain API Calls
2. Fraudulent Access Token Use
3. Information Leaks
4. User Swapping

MCP Elicitation in URL Mode: The Risk of User Swapping

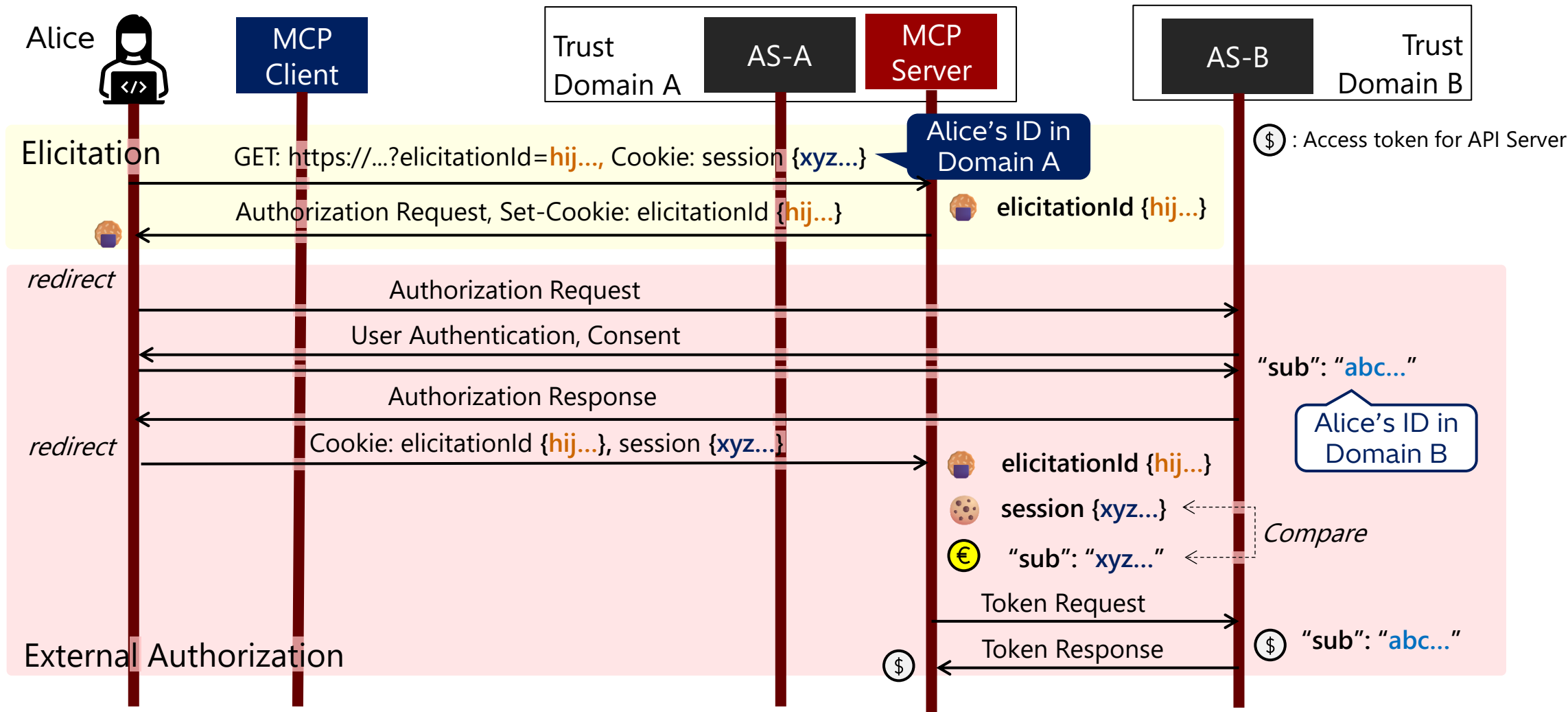


Countermeasure*: Cookie Comparison



*: I worked out these details based on <https://modelcontextprotocol.io/specification/2026-07-28/client/elicitation#phishing>

Countermeasure*: Cookie Comparison



*: I worked out these details based on <https://modelcontextprotocol.io/specification/2026-07-28/client/elicitation#phishing>

Countermeasure*: Cookie Comparison

Challenges:

- A browser cannot handle multiple elicitation simultaneously.
- The AS needs to implement such a cookie.
- The cookie should be configured to be reached to MCP Server, which means that whenever a new MCP Server is added, then the configuration might be updated.

MCP Elicitation in URL Mode - The Risk of User Swapping

Assumption: MCP Authorization follows security practices, so it is secure.

In external authorization,



Risk: Low

- Phishing Aimed at User Swapping in Elicitation: Type 1
 - ✓ Attacker's Capability: Make victim access MCP Server's elicitation start page
- Phishing Aimed at User Swapping in Elicitation: Type 2
 - ✓ Attacker's Capability: Make victim sends an authorization request of the external authorization
- Phishing Aimed at User Swapping in Elicitation: Type 3
 - ✓ Attacker's Capability: Make victim sends a modified authorization request of the external authorization.
 - ✓ Vulnerabilities: The external authorization does not follow OAuth2 security practices.

Risk: High

Phishing Aimed at User Swapping in Elicitation: Type 1

Threat Model:

Attacker's Capability:

Make Alice access MCP
Server's elicitation start page

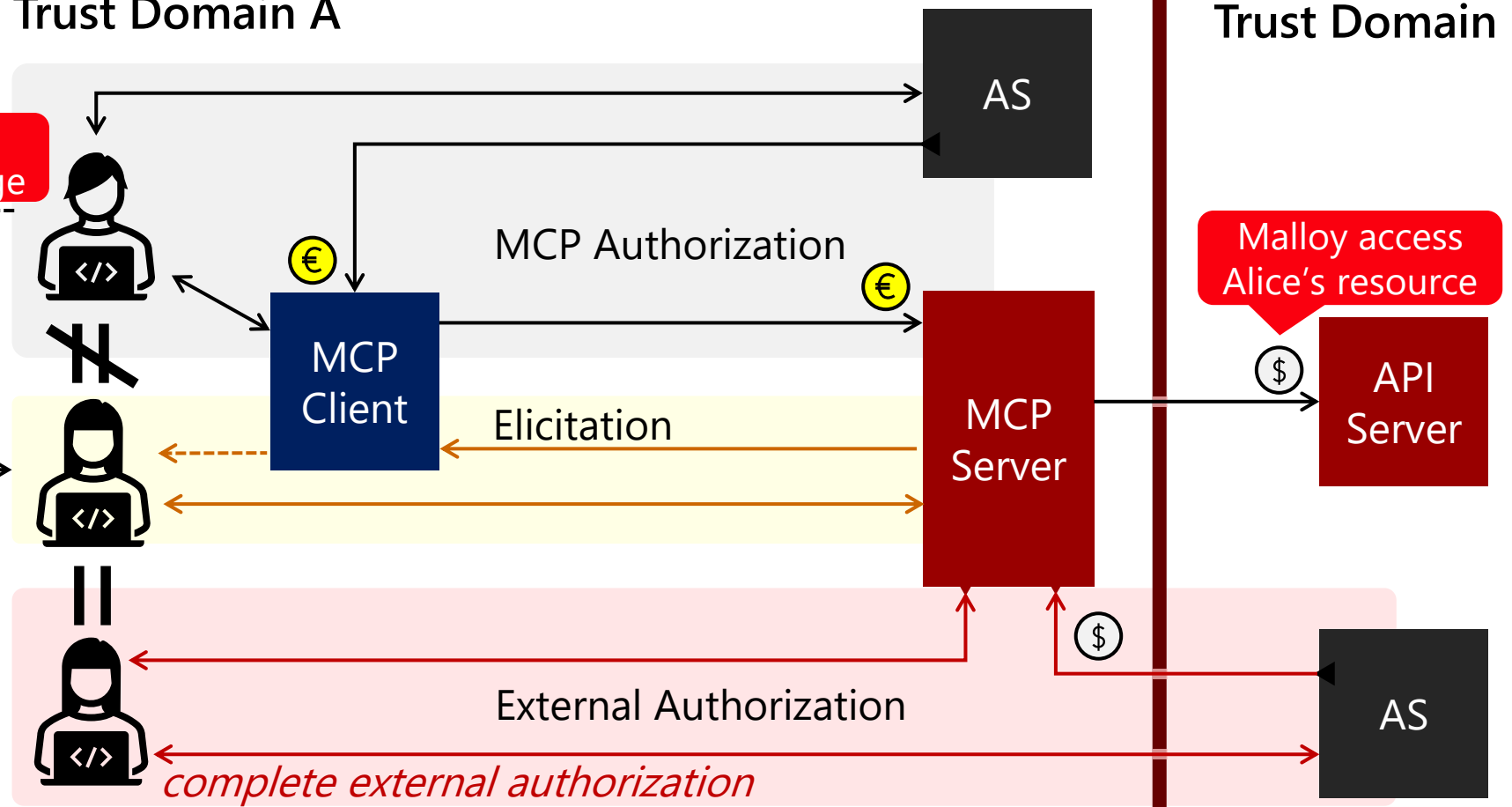
User/Agent
"Mallory"

User/Agent
"Alice"

User/Agent
"Alice"

Trust Domain A

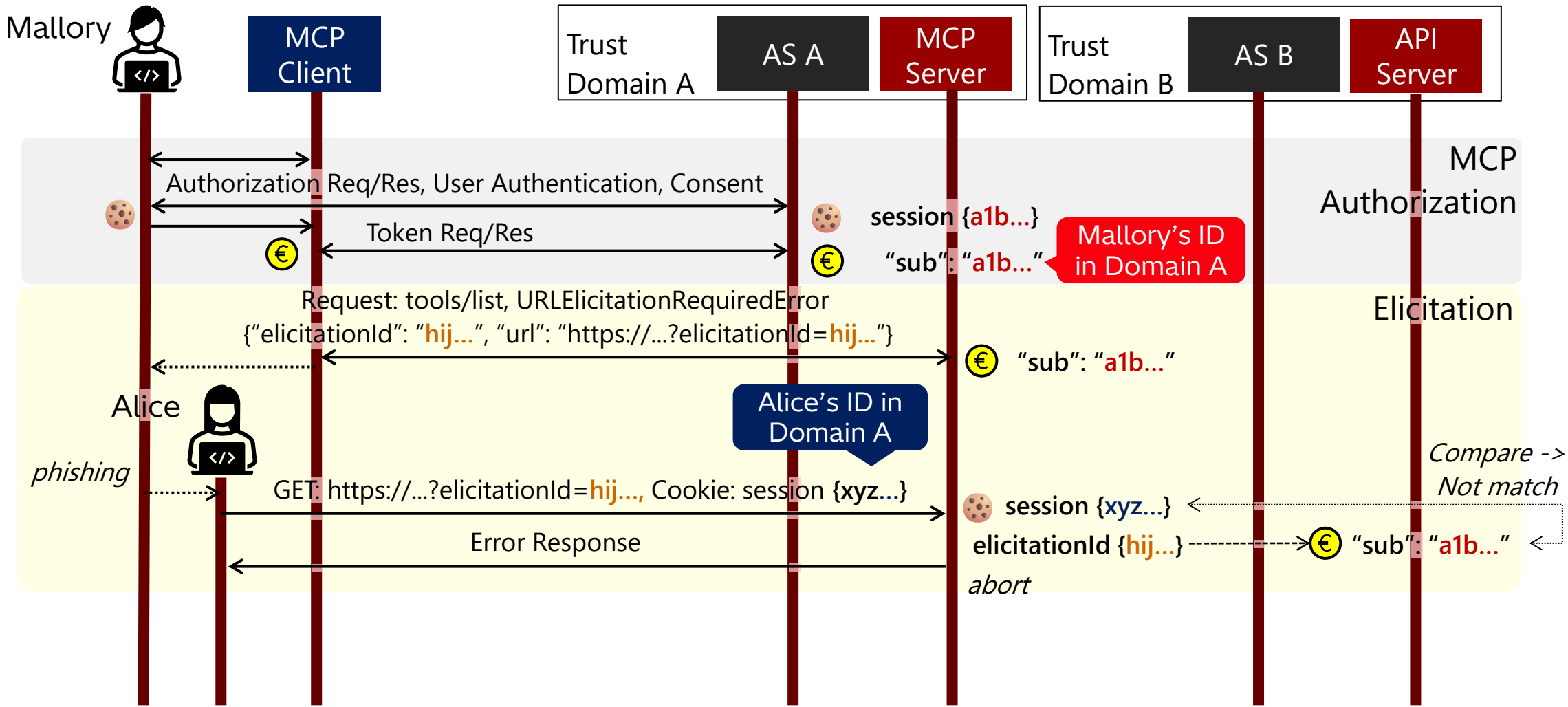
Trust Domain B



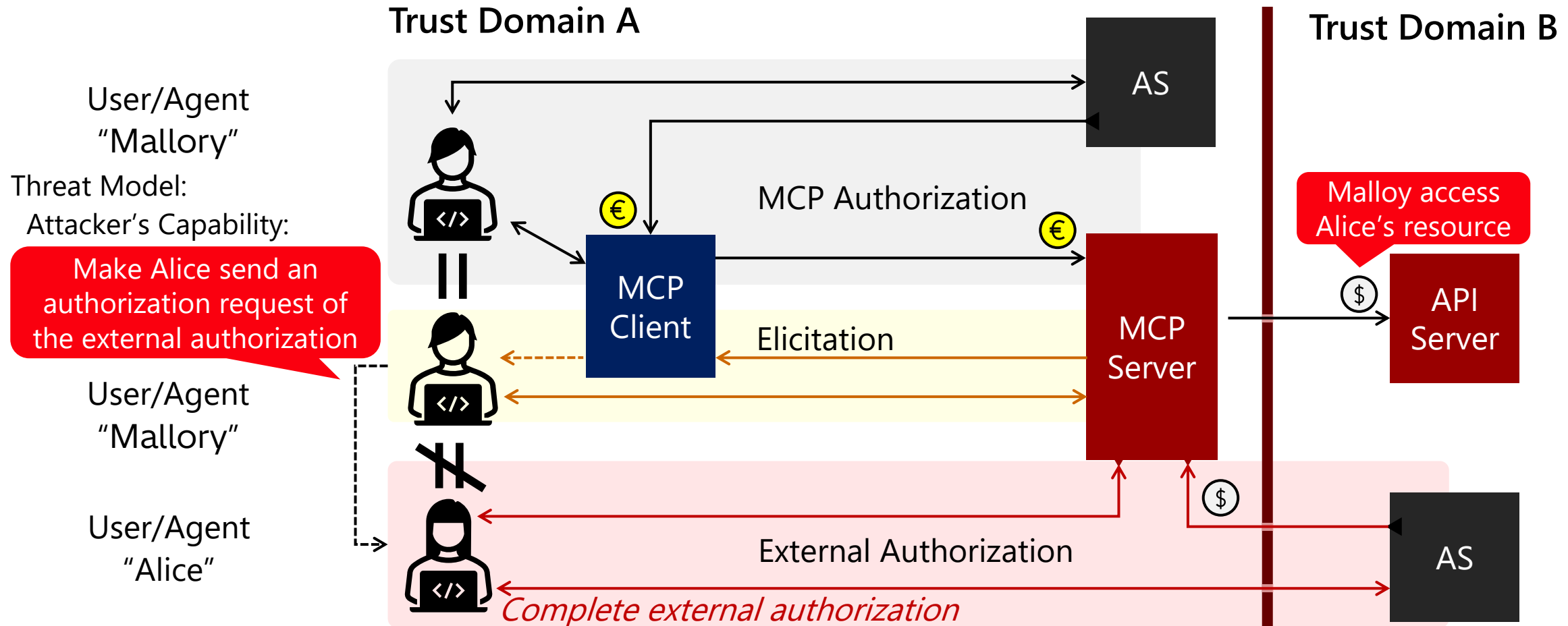
€ : Access Token for MCP Server in Domain A

\$: Access Token for API Server in Domain B

Phishing Aimed at User Swapping in Elicitation: Type 1



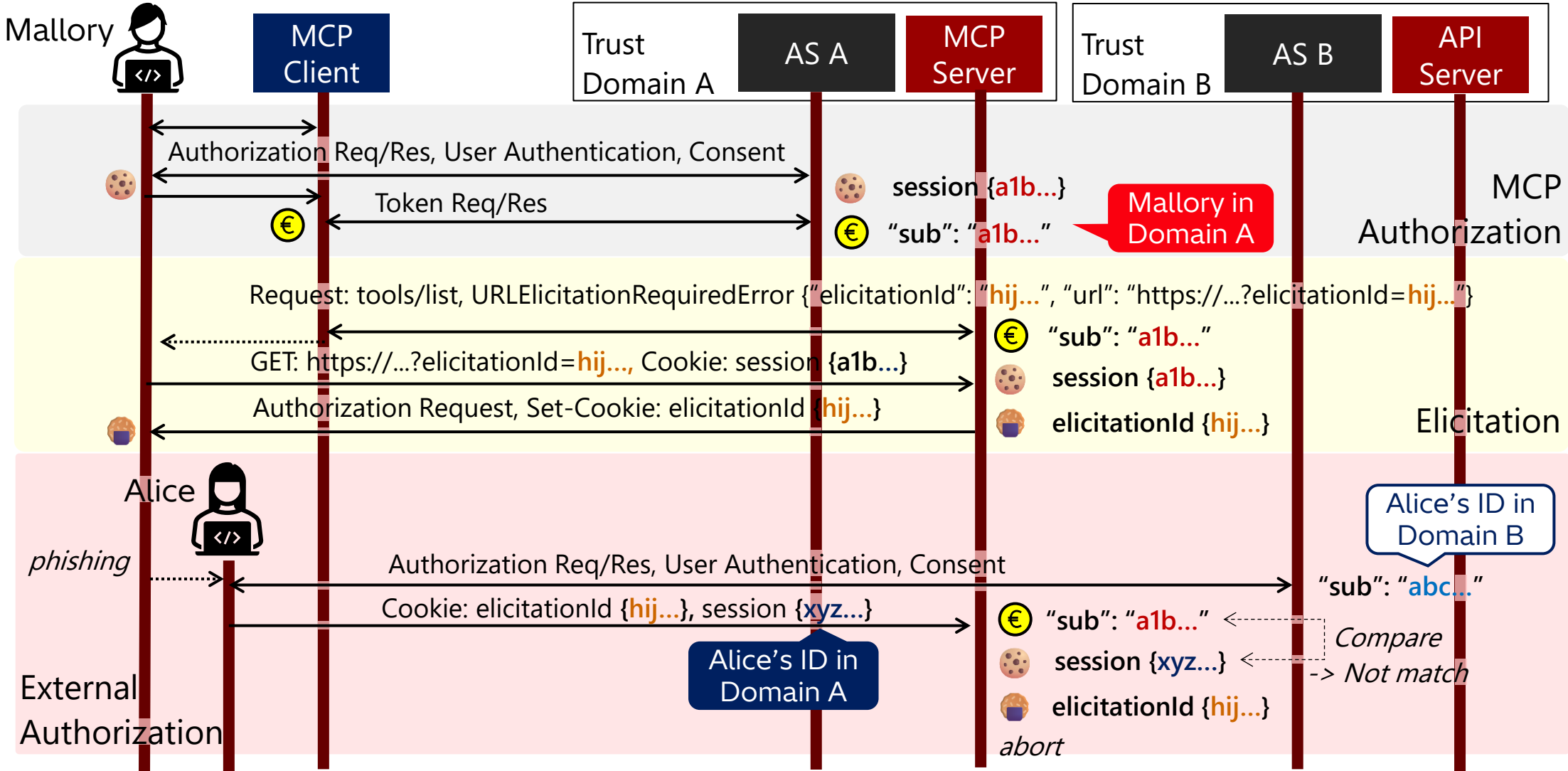
Phishing Aimed at User Swapping in Elicitation: Type 2



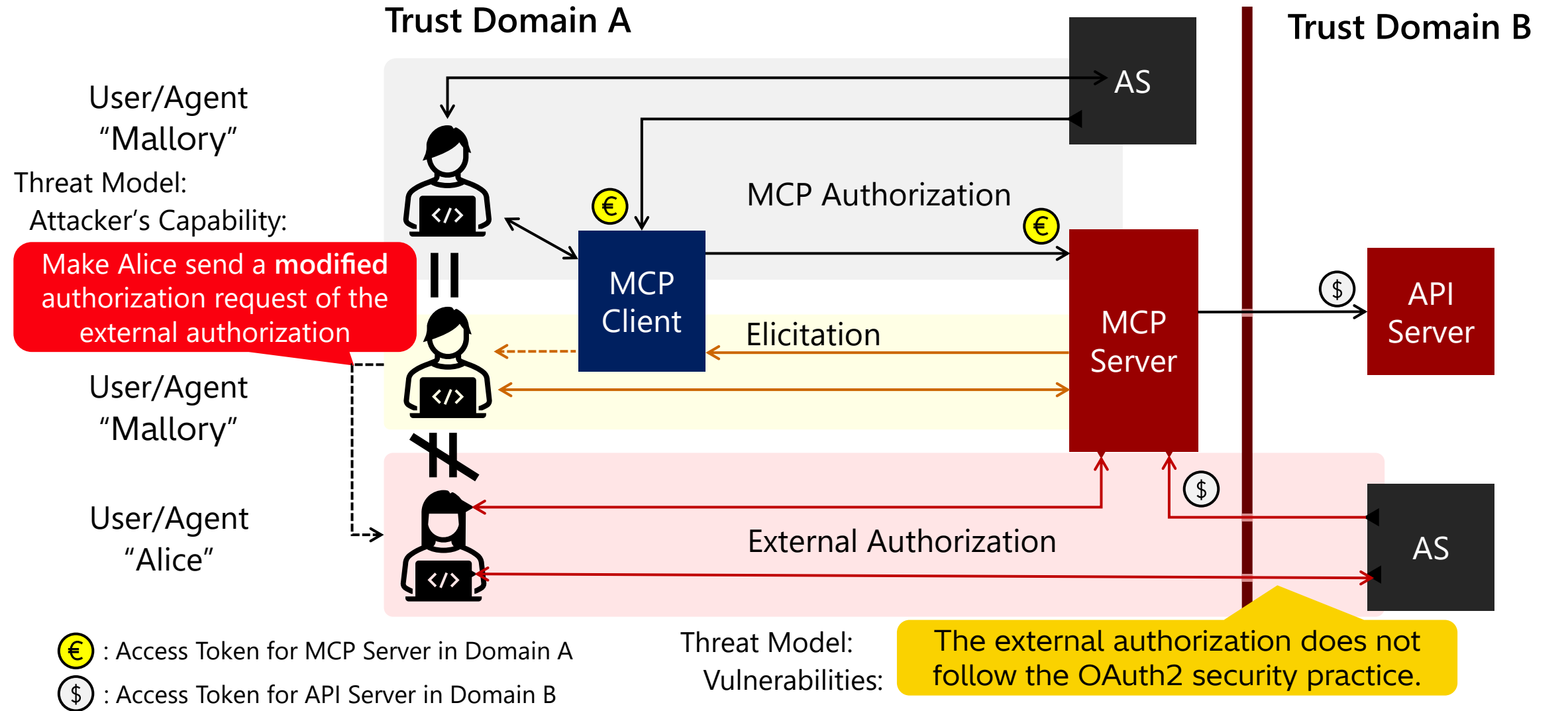
€ : Access Token for MCP Server in Domain A

\$: Access Token for API Server in Domain B

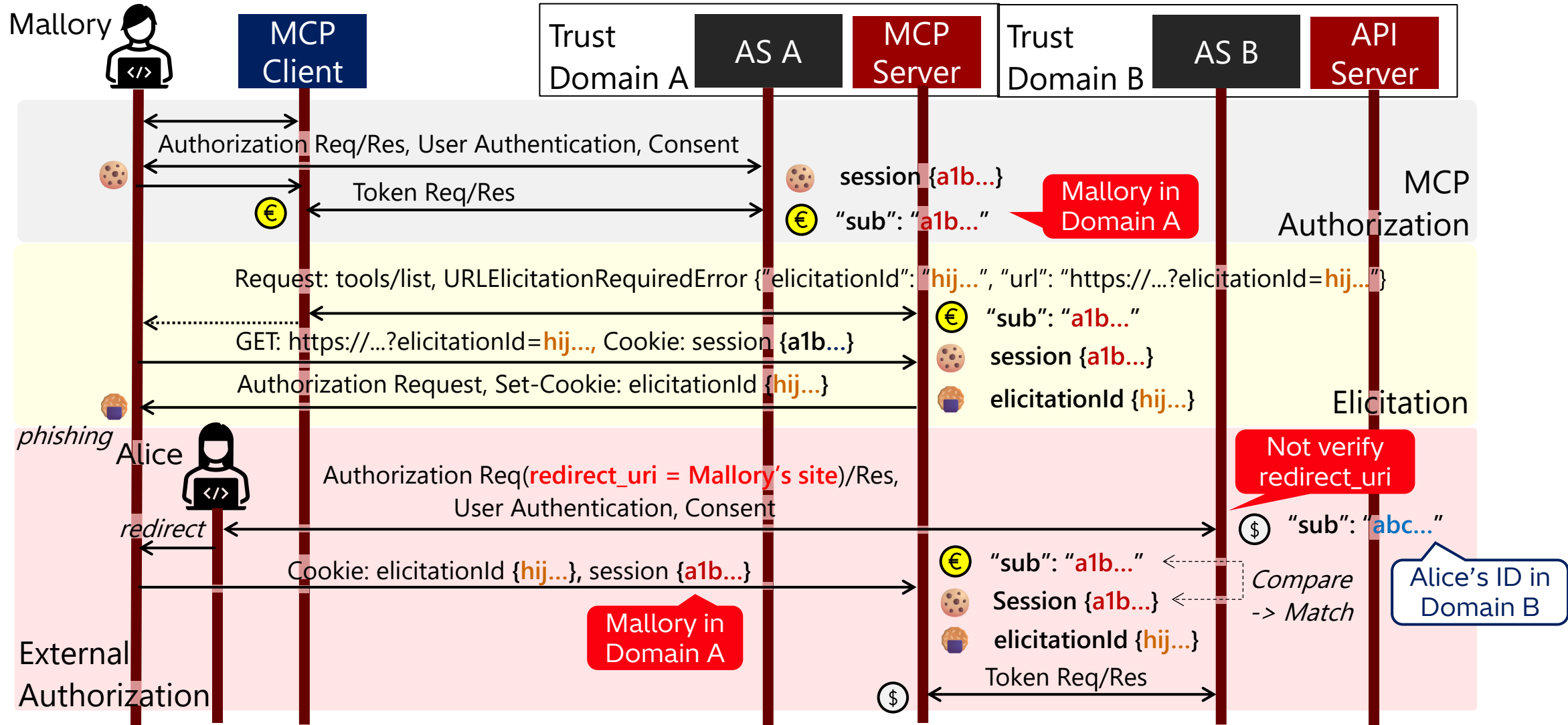
Phishing Aimed at User Swapping in Elicitation: Type 2



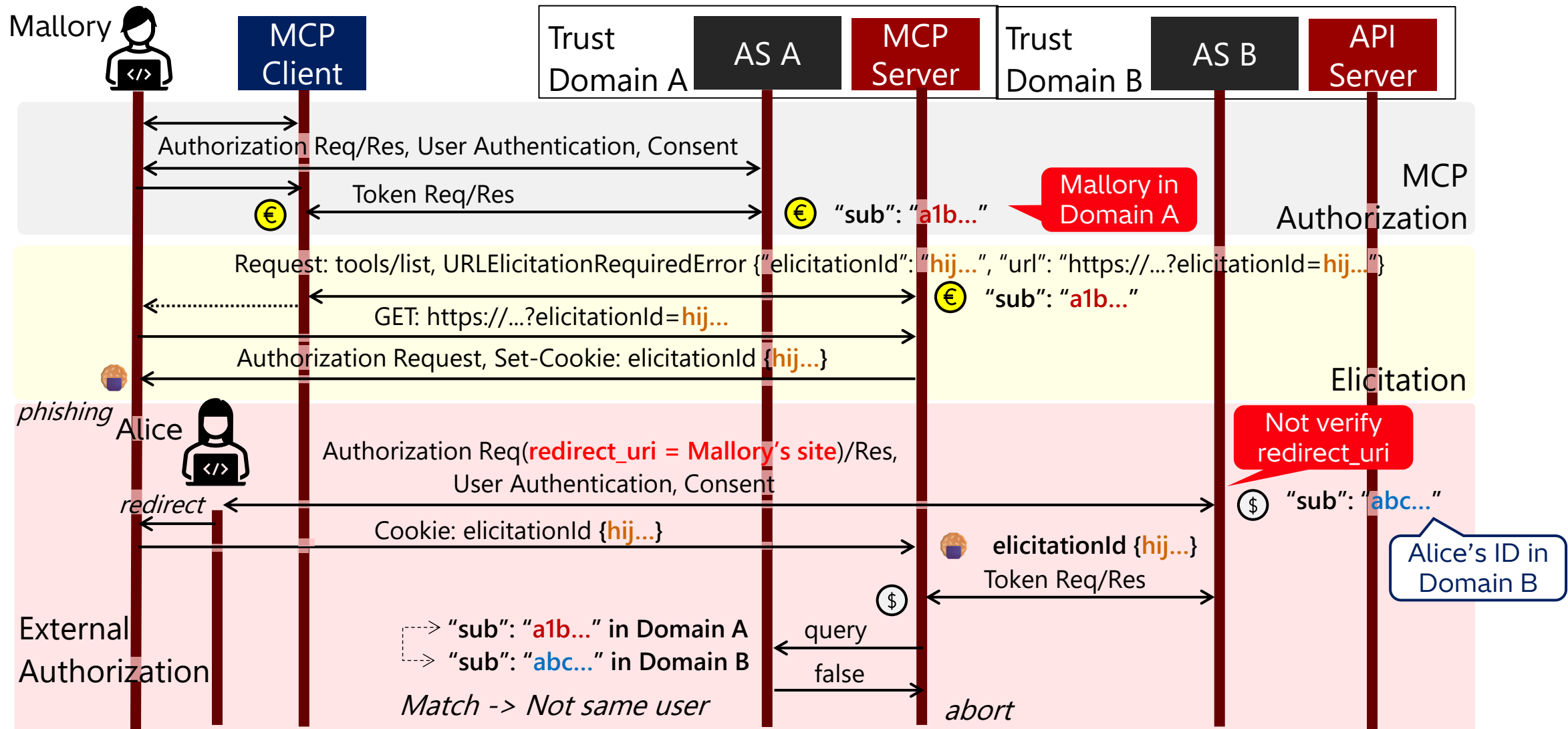
Phishing Aimed at User Swapping in Elicitation: Type 3



Phishing Aimed at User Swapping in Elicitation: Type 3



Countermeasure: User ID Comparison



Countermeasure: User ID Comparison

Fact:

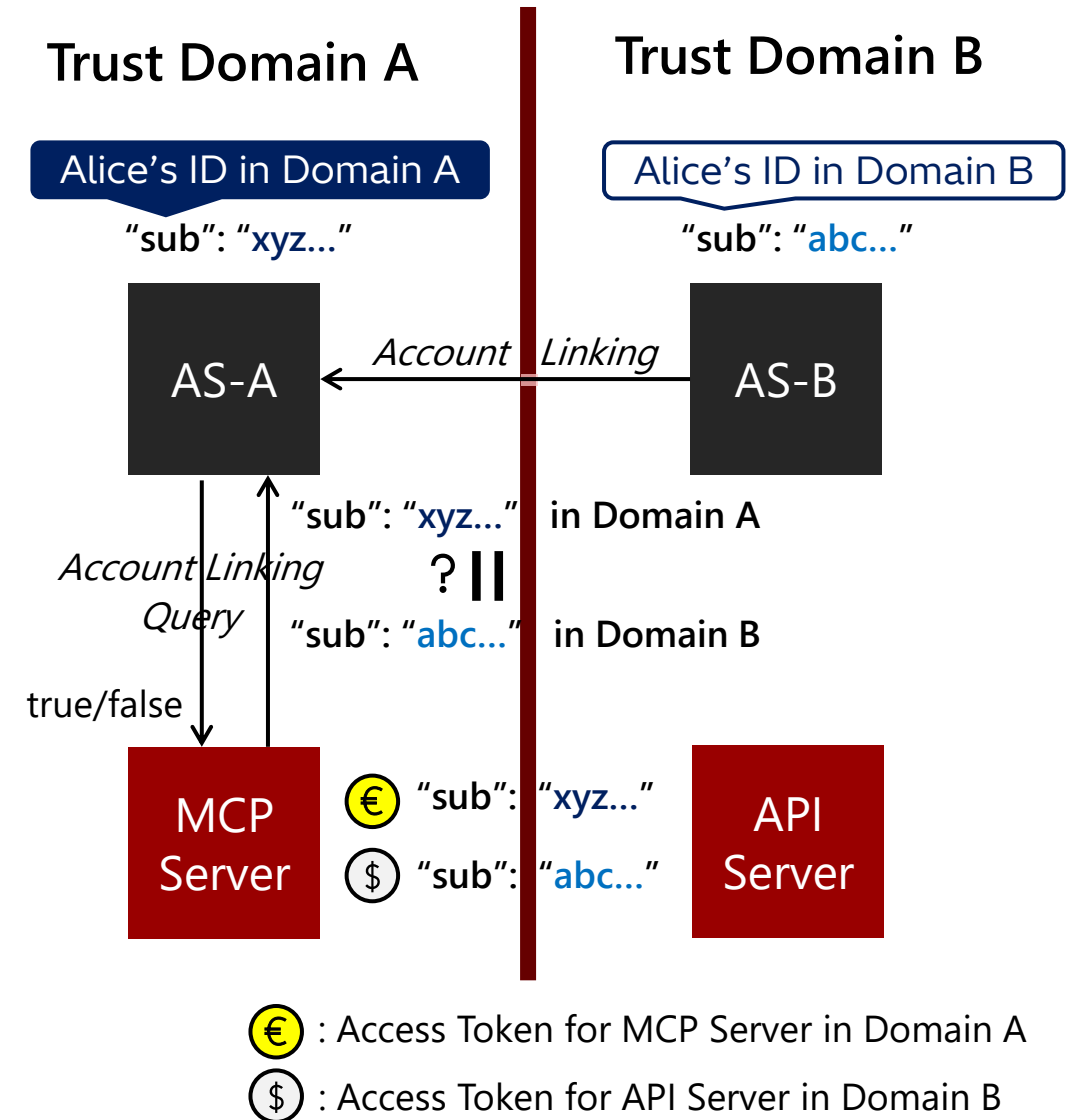
- Same user, different ID in different domain.
- MCP Server does not know that.
- No standard way for AS to know that while there are several ways can be considered.

Ex. Account linking:

- User explicitly register to AS in Domain A their ID registered in AS in Domain B.
- During identity federation/social login, ID registered in AS in Domain B is registered to AS in Domain A.

Solution: AS implements the following query service:

- MCP Server sends user IDs in both Domains.
- AS returns if they are the same user's ID or not.



Countermeasure: User ID Comparison

Challenges:

- There is no standard for the account linking.
- There is no standard for the query.
- Privacy concerns: Possible user ID correlation among domains, which allows many domains to build a much more comprehensive picture of the user's behavior.

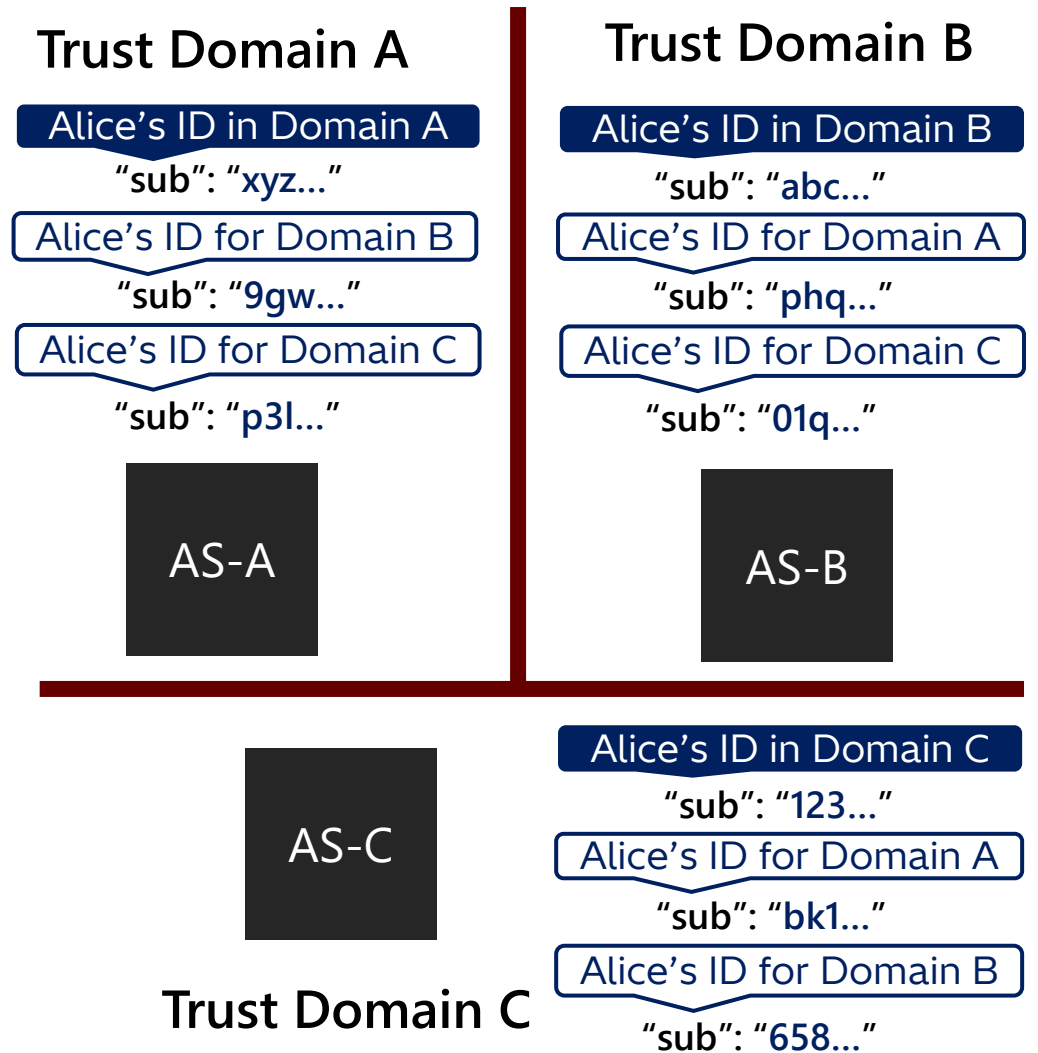
Remedies:

- Privacy concerns: use Pairwise Pseudonymous Identifier* (PPID) for "sub" claim.

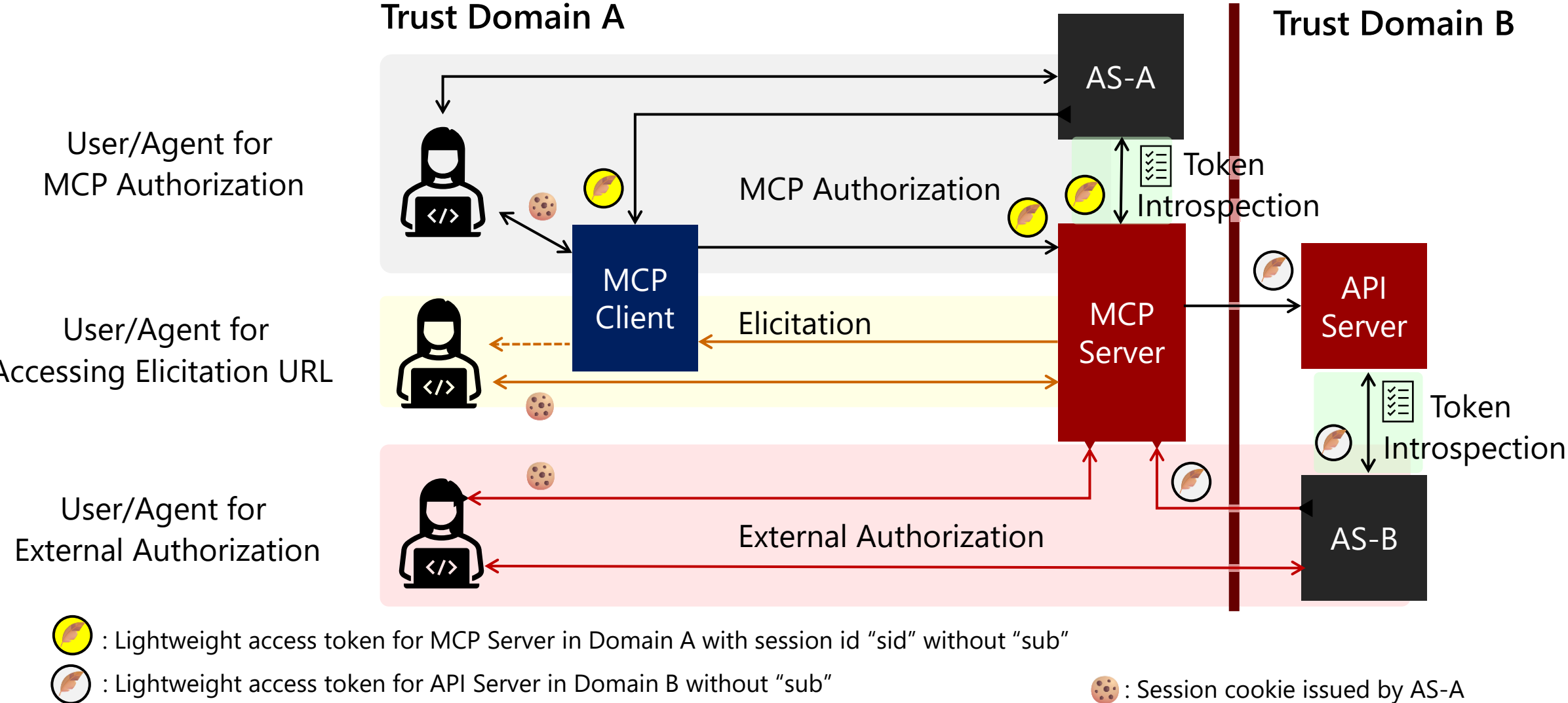
Practical Alternatives:

- Apply Cookie Comparison **without User ID** and only accept **Domain assuring secure external authorization**.

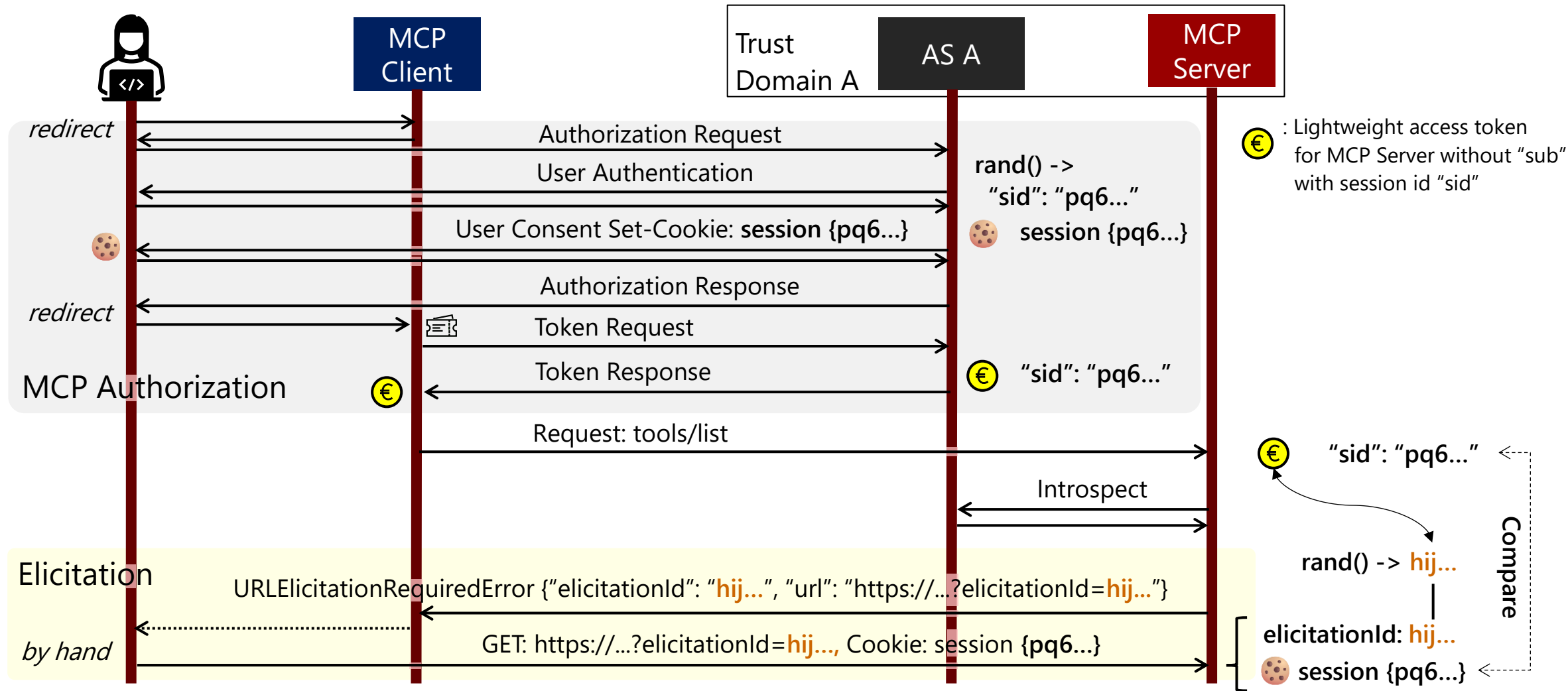
*: https://openid.net/specs/openid-connect-core-1_0.html#SubjectIDTypes



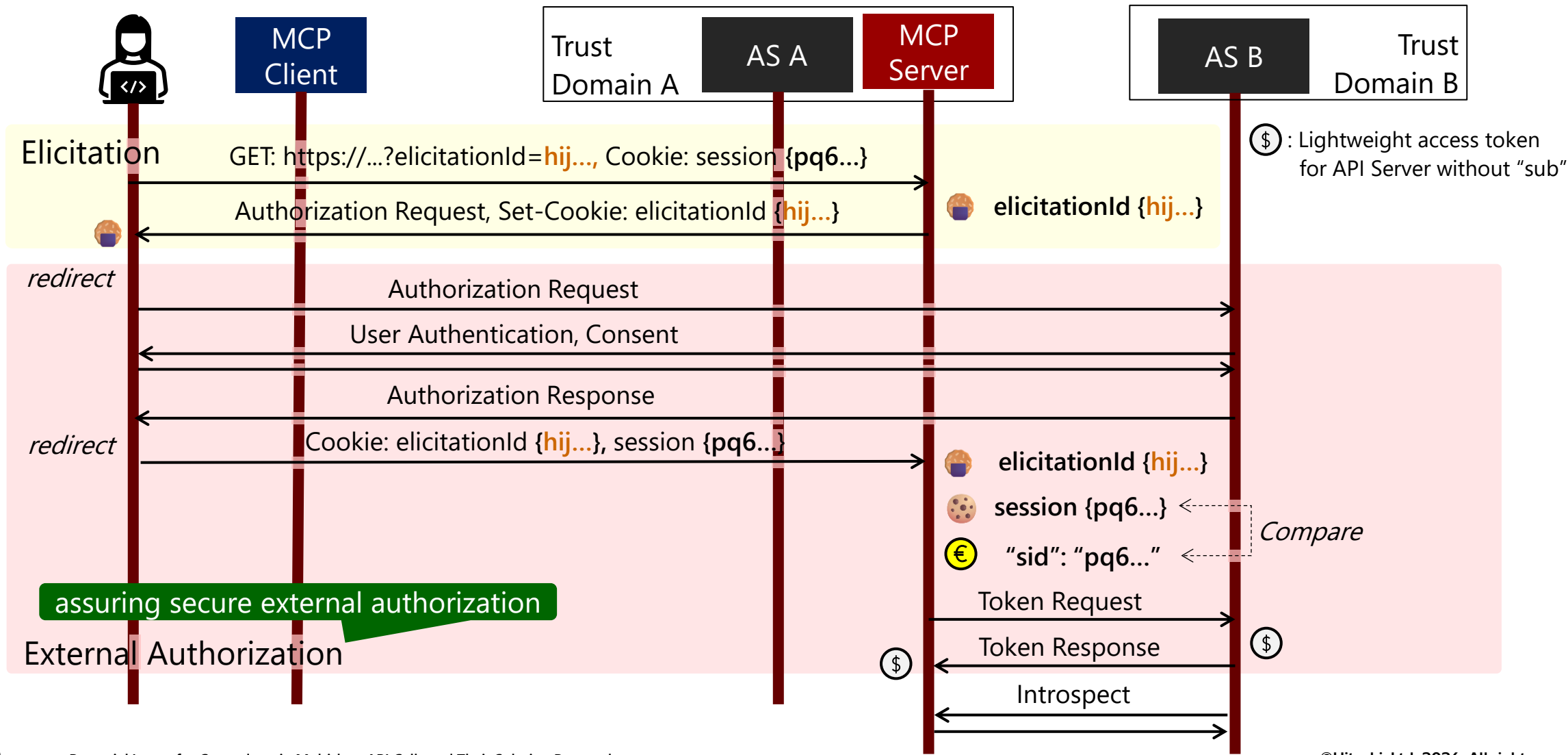
Countermeasure: Cookie Comparison without User ID



Countermeasure: Cookie Comparison without User ID



Countermeasure: Cookie Comparison without User ID



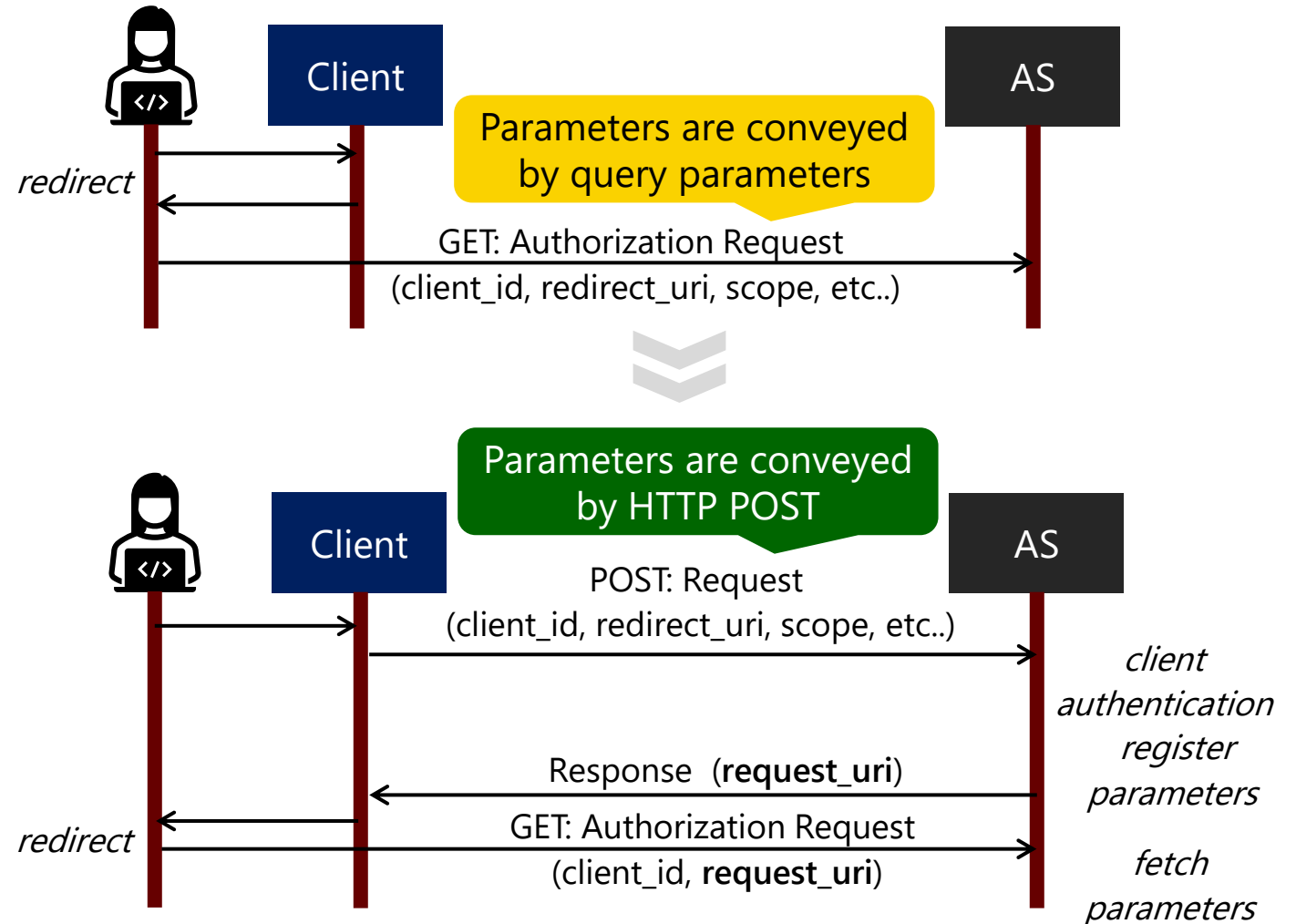
Secure External Authorization: Pushed Authorization Request^{*1}

The contents of an authorization request are pushed to AS.

The authorization request only include "client_id" and "request_uri", which is a key to fetch the contents registered inside AS.

[Effect]

- An attacker cannot modify parameters like "redirect_uri" the request.
- The attacker cannot impersonate a client to register its authorization request contents.



^{*1}: RFC 9126 OAuth 2.0 Pushed Authorization Requests (<https://www.rfc-editor.org/rfc/rfc9126.html>)

Challenges in Practice

Assumption for Practitioners:

- You want to apply security practices for secure MCP authorization.
- You want to federate only with Domain assuring secure external authorization.

In order to do so...

- You can control all entities in Domain A.

However...

- You cannot control all entities in Domain B.

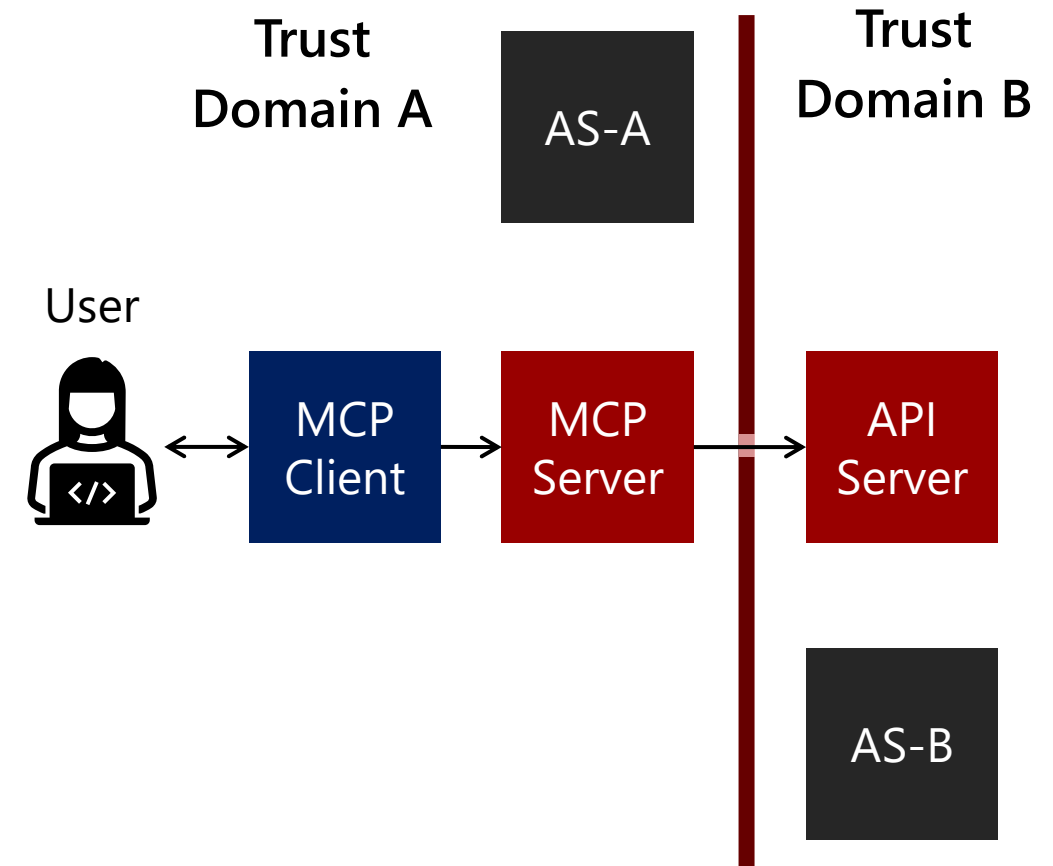
But...

- You can evaluate Domain B's authorization security (but how?).

➔ Investigate their documentation and metadata*

➔ Try an external authorization.

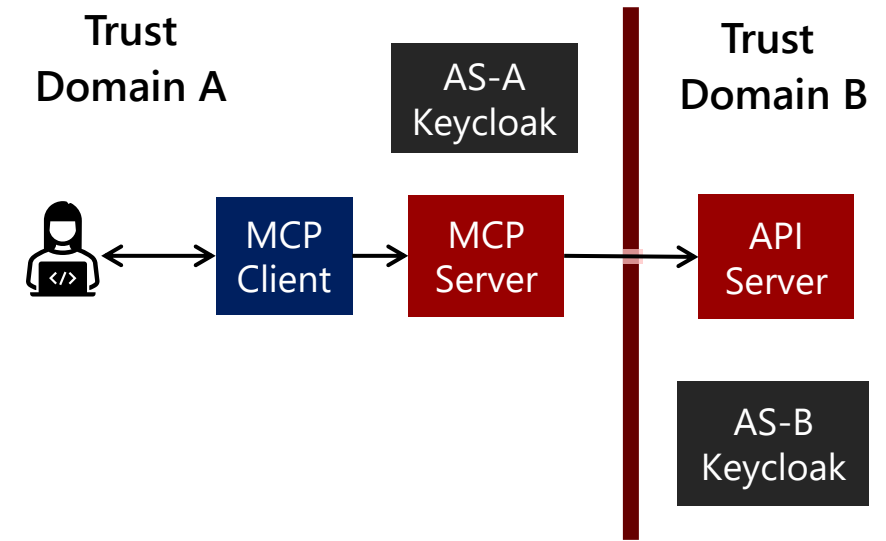
! but not interfere with Domain B's service



* IANA OAuth Parameters list up OAuth-related parameters.
(<https://www.iana.org/assignments/oauth-parameters/oauth-parameters.xhtml>)

Implementation Example: Keycloak

- Supported Specifications:
<https://www.keycloak.org/securing-apps/specifications>
- MCP Authorization:
<https://www.keycloak.org/securing-apps/mcp-authz-server>
- Token Exchange:
<https://www.keycloak.org/securing-apps/token-exchange>
- JWT Authorization Grant:
<https://www.keycloak.org/securing-apps/jwt-authorization-grant>
- OAuth Identity and Authorization Chaining Across Domains:
<https://www.keycloak.org/securing-apps/oauth-identity-authorization-chaining-across-domains>
- Identity Assertion JWT Authorization Grant (ID-JAG):
<https://www.keycloak.org/securing-apps/identity-assertion-jwt-authorization-grant>
- OAuth 2.0 Demonstrating Proof of Possession (DPoP):
<https://www.keycloak.org/securing-apps/dpop>
- Lightweight Access Token:
https://www.keycloak.org/docs/latest/server_admin/index.html#_using_lightweight_access_token



Key Takeaways

- For secure cross-domain multi-hop API calls, forwarding an access token and token exchange only have some issues.
- Instead, OAuth Identity and Authorization Across Domains or MCP Elicitation in URL mode does not have such issues but has other potential issues.
- To resolve the issues, some additional security measures can be applied: using session cookie, user ID comparison.
- However, whether the measures can be applied depends on the security of different domains. Therefore, the different domain's security needs to be investigated in advance.

Trademarks

- OpenID is a trademark or registered trademark of OpenID Foundation in the United States and other countries.
- Other brand names and product names used in this material are trademarks, registered trademarks, or trade names of their respective holders.

Thank you

AGNTCon + MCPCon Europe 2026
@RAI Amsterdam Convention Centre, Amsterdam, Netherlands

Potential Issues for Cross-domain Multi-hop API Calls and Their Solution Proposal

Takashi Norimatsu

Date

Open Source Program Office (OSPO)

September 18, 2026

Hitachi, Ltd.

Appendix: Metadata Directory

Authorization Server Metadata (AS in Domain A)

OAuth Identity and Authorization Chaining Across Domains

Parameter	Description	Values	Description
grant_types_supported ^{*1}	JSON array containing a list of the OAuth 2.0 grant type values that this authorization server supports	urn:ietf:params:oauth:grant-type:token-exchange ^{*2}	Token exchange grant type for OAuth 2.0
identity_chaining_requested_token_types_supported ^{*3}	JSON array containing a list of Token Type Identifiers supported as a requested_token_type in an Identity and Authorization Chaining Token Exchange request.	urn:ietf:params:oauth:token-type:access_token ^{*2}	Token type URI for an OAuth 2.0 access token

*1: RFC 8414 OAuth 2.0 Authorization Server Metadata (<https://datatracker.ietf.org/doc/html/rfc8414#section-2>)

*2: RFC 8693 OAuth 2.0 Token Exchange (<https://datatracker.ietf.org/doc/html/rfc8693#section-3>)

*3: Internet Draft: OAuth Identity and Authorization Chaining Across Domains (<https://www.ietf.org/archive/id/draft-ietf-oauth-identity-chaining-17.html#section-3>)

Authorization Server Metadata (AS in Domain B)

OAuth Identity and Authorization Chaining Across Domains

Parameter	Description	Values	Description
grant_types_supported* ¹	JSON array containing a list of the OAuth 2.0 grant type values that this authorization server supports	urn:ietf:params:oauth:grant-type:jwt-bearer* ²	JWT Bearer Token Grant Type Profile for OAuth 2.0
		urn:ietf:params:oauth:grant-type:jwt-dpop* ³	DPoP-bound JWT Authorization Grant

RFC 9126 OAuth 2.0 Pushed Authorization Requests (PAR):

Parameter	Description
pushed_authorization_request_endpoint* ⁴	URL of the authorization server's pushed authorization request endpoint.

*1: RFC 8414 OAuth 2.0 Authorization Server Metadata (<https://datatracker.ietf.org/doc/html/rfc8414#section-2>)

*2: RFC 7523 JSON Web Token (JWT) Profile for OAuth 2.0 Client Authentication and Authorization Grants (<https://datatracker.ietf.org/doc/html/rfc7523#section-8.1>)

*3: Individual Internet Draft: OAuth 2.0 JWT Authorization Grant with DPoP Binding (<https://www.ietf.org/archive/id/draft-parecki-oauth-jwt-dpop-grant-01.html#section-3.1>)

*4: RFC 9126 OAuth 2.0 Pushed Authorization Requests (<https://www.rfc-editor.org/rfc/rfc9126.html#section-5>)

Authorization Server Metadata (AS in Domain B)

Proof Key for Code Exchange (PKCE):

Parameter	Description	Values
code_challenge_methods_supported ^{*1}	PKCE code challenge methods supported by this authorization server	plain ^{*2} S256 ^{*2}

OAuth 2.0 Demonstrating Proof of Possession (DPoP):

Parameter	Description	Values
dpop_signing_alg_values_supported ^{*3}	JSON array containing a list of the JWS algorithms supported for DPoP proof JWTs	ES256, PS256, EdDSA, etc.

RFC 9207 OAuth 2.0 Authorization Server Issuer Identification:

Parameter	Description	Values
authorization_response_iss_parameter_supported ^{*4}	Boolean value indicating whether the authorization server provides the iss parameter in the authorization response.	true/false

^{*1}: RFC 8414 OAuth 2.0 Authorization Server Metadata (<https://datatracker.ietf.org/doc/html/rfc8414#section-2>)

^{*2}: RFC 7636 Proof Key for Code Exchange by OAuth Public Clients (<https://datatracker.ietf.org/doc/html/rfc7636#section-4.2>)

^{*3}: RFC 9449 OAuth 2.0 Demonstrating Proof of Possession (DPoP) (<https://datatracker.ietf.org/doc/html/rfc9449#section-5.1>)

^{*4}: RFC 9207 OAuth 2.0 Authorization Server Issuer Identification (<https://datatracker.ietf.org/doc/html/rfc9207#section-3>)

Client Metadata (MCP Server in Domain A)

OAuth Identity and Authorization Chaining Across Domains

Parameter	Description	Values	Description
grant_types ^{*1}	JSON array containing a list of the OAuth 2.0 grant type values that this authorization server supports	urn:ietf:params:oauth:grant-type:token-exchange ^{*2}	Token exchange grant type for OAuth 2.0
		urn:ietf:params:oauth:grant-type:jwt-bearer ^{*3}	JWT Bearer Token Grant Type Profile for OAuth 2.0

OAuth 2.0 Demonstrating Proof of Possession (DPoP):

Parameter	Description	Values
dpop_bound_access_tokens ^{*4}	Boolean value showing whether the client always uses DPoP for token requests	true/false

*1: RFC 7591 OAuth 2.0 Dynamic Client Registration Protocol (<https://datatracker.ietf.org/doc/html/rfc7591#section-2>)

*2: RFC 8693 OAuth 2.0 Token Exchange (<https://datatracker.ietf.org/doc/html/rfc8693#section-3>)

*3: RFC 7523 JSON Web Token (JWT) Profile for OAuth 2.0 Client Authentication and Authorization Grants (<https://datatracker.ietf.org/doc/html/rfc7523#section-8.1>)

*4: RFC 9449 OAuth 2.0 Demonstrating Proof of Possession (DPoP) (<https://datatracker.ietf.org/doc/html/rfc9449#section-5.2>)

Client Metadata (MCP Server in Domain A)

RFC 9126 OAuth 2.0 Pushed Authorization Requests (PAR):

Parameter	Description	Values
require_pushed_authorization_requests* ¹	Boolean parameter indicating whether the only means of initiating an authorization request the client is allowed to use is PAR. If omitted, the default value is false.	true/false

*1: RFC 9126 OAuth 2.0 Pushed Authorization Requests (<https://www.rfc-editor.org/rfc/rfc9126.html#section-6>)

Protected Resource Metadata (API Server in Domain B)

OAuth 2.0 Demonstrating Proof of Possession (DPoP):

Parameter	Description	Values
dpop_signing_alg_values_supported* ¹	JSON array containing a list of the JWS alg values supported by the resource server for validating DPoP proof JWTs	ES256, PS256, EdDSA, etc.
dpop_bound_access_tokens_required* ¹	Boolean value specifying whether the protected resource always requires the use of DPoP-bound access tokens	true/false

*1: RFC 9728 OAuth 2.0 Protected Resource Metadata (<https://datatracker.ietf.org/doc/html/rfc9728#section-2>)

HITACHI